

Defense Against LLM Backdoors using Critical Neuron Isolation Pruning

Yuxi Li
Huazhong University of Science and
Technology
China
yuxili@hust.edu.cn

Zhibo Zhang
Huazhong University of Science and
Technology
China
zhangzhibom@hust.edu.cn

Kailong Wang*
Huazhong University of Science and
Technology
China
wangkl@hust.edu.cn

Xingshuo Han
Nanjing University of Aeronautics
and Astronautics
China
xingshuo.han@nuaa.edu.cn

Ling Shi
Nanyang Technological University
Singapore
ling.shi@ntu.edu.sg

Haoyu Wang
Huazhong University of Science and
Technology
China
haoyuwang@hust.edu.cn

Abstract

Large language models (LLMs) have advanced rapidly across domains, yet their growing complexity increases vulnerability to security threats such as *backdoor attacks*, where hidden triggers induce malicious or unintended outputs. Existing defenses generally fall into inference-time detection or training-time mitigation, yet they face two fundamental limitations. First, they are primarily designed for fine-tuning-based backdoors, particularly those embedded in PEFT modules, and therefore fail to address more insidious model-editing attacks that bypass conventional training pipelines. Second, they are typically developed around simple classification settings and do not naturally extend to the open-ended generation characteristics of LLMs. Consequently, these methods focus on surface-level behavioral patterns while neglecting the deeper representational causes of malicious activations. This lack of mechanistic understanding forces defenses to depend on empirical heuristics, limiting their robustness, generality, and practical applicability in real-world LLM deployment.

To bridge this gap, we introduce DeCNIP (**Defense with Critical Neuron Isolation Pruning**). It leverages representational analysis to identify and neutralize backdoors within a single detection and mitigation pipeline. Specifically, DeCNIP identifies trigger-like behaviors by optimizing a cross-entropy-based loss between harmful prompts with candidate tokens and benign inputs. This deep representational discovery enables the framework to expose latent threats by uncovering the fundamental mechanisms through which triggers hijack model weights. It then isolates **Backdoor Critical Neurons (BCNs)** and prunes them selectively to remove malicious influence while preserving model utility. Extensive evaluations on

six open-source LLMs and two benchmark datasets demonstrate that DeCNIP achieves **more than 95% relative reduction in Attack Success Rate (ASR)**, outperforming seven state-of-the-art defenses with **only 0.1% of the neurons intervened**. Moreover, it maintains **an average of 97% of the model's foundational performance** on normal benchmarks, illustrating its efficacy, robustness, and scalability in securing large-scale generative models.

CCS Concepts

- **Security and privacy** → **Software and application security**;
- **Computing methodologies** → **Natural language processing**.

ACM Reference Format:

Yuxi Li, Zhibo Zhang, Kailong Wang, Xingshuo Han, Ling Shi, and Haoyu Wang. 2026. Defense Against LLM Backdoors using Critical Neuron Isolation Pruning. In . ACM, New York, NY, USA, 21 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Large language models (LLMs) have advanced rapidly in recent years and now play a critical role across a wide range of domains, including industry, education, and healthcare [3, 33, 37, 49]. Nevertheless, these models face severe security threats from backdoor attacks [13, 20, 42], which involve the implantation of hidden malicious logic that remains dormant until activated by specific input triggers. Backdoor techniques targeting LLMs primarily fall into two categories: fine-tuning-based attacks, where attackers inject poisoned samples into the training pipeline, and model-editing attacks, which involve direct manipulation of model weights to embed trigger-response pairs. These attacks severely undermine model integrity, potentially leading to unauthorized data exfiltration or the generation of harmful content.

Existing defenses against backdoor attacks can be broadly divided into *inference-time defense* and *training-time defense* [2, 19]. Inference-time detection aims to identify and suppress abnormal model behaviors during inference by analyzing the relationship between inputs and outputs [30, 32]. However, this class of defenses faces two major limitations. First, many existing methods are specifically tailored to identify backdoor artifacts within PEFT modules and LoRA adapters. Consequently, they often fail to generalize to attacks via model editing, which bypasses traditional fine-tuning

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

pipelines by directly manipulating model weights, thereby significantly narrowing the practical detection scope. Second, most detection mechanisms focus exclusively on the surface-level behaviors of the model, neglecting the deeper representational causes that drive such malicious activations.

Compared to inference-time defenses, training-time defenses attempt to neutralize backdoors before deployment by adjusting models' parameters, often through fine-tuning or model pruning. Fine-tuning-based defenses generally fail to completely remove backdoor behaviors, leaving residual vulnerabilities that attackers may still exploit. Meanwhile, pruning-based defenses effectively address the root causes of backdoors but still encounter two fundamental challenges in the context of generative LLMs. On one hand, although localizing backdoor neurons and pruning have been explored for backdoor removal in conventional deep learning models [21, 23, 40], these methods are primarily designed for discriminative tasks with a closed-set label space (e.g., image or text classification). Directly transferring them to decoder-only LLMs is difficult because the nature of backdoor triggers differs significantly between classification tasks and generative language modeling.

On the other hand, while recent pruning strategies have been proposed for language models [6, 46], they predominantly target encoder-only architectures (e.g., BERT, RoBERTa) where the defense objective is to rectify a flipped classification label. In contrast, backdoors in generative LLMs hijack the entire autoregressive trajectory, making the identification of "malicious neurons" significantly more complex and computationally expensive. Compounding this challenge is the fact that prior pruning research for LLMs has been largely optimized for inference acceleration [31, 50] rather than robustness. Consequently, these methods often disregard the safety-critical neurons that govern malicious activations. Applying them directly thus causes a severe drop in model utility and stability, as they fail to preserve the balance between security and the model's inherent reasoning capabilities. These limitations motivate a key question: *Can we design an approach that effectively and surgically removes backdoors in decoder-only generative LLMs while preserving the model's original performance and complex reasoning capabilities?*

To establish a principled foundation for defense, we conduct a systematic analysis to characterize the operational mechanisms of backdoors during inference. By evaluating the model's response to varied trigger configurations and analyzing internal hidden state evolutions across benign and harmful contexts, we ensure a rigorous assessment of how malicious logic is activated. Our investigation reveals two pivotal insights: first, we observe that the trigger's influence on model behavior is largely invariant to its specific surface form or spatial positioning within a prompt, with malicious behavior being consistently elicited across diverse token variations and locations; second, layer-wise activation analysis demonstrates that harmful prompts containing these triggers generate internal representations that are deceptively similar to those of benign queries. *These findings collectively suggest that generative backdoors are not mere surface-level mappings but are encoded in deep representational structures that hijack the model's reasoning trajectory by mimicking benign processing patterns.*

Motivated by the insight that triggers function as mechanistic "entry points" within the representational space, we propose

DECNIP, a unified framework designed to interpret and neutralize backdoors through deep activation analysis. The framework utilizes a cross-entropy-based optimization objective that compares augmented harmful prompts against benign counterparts, which enables the discovery of latent triggers by exposing the fundamental mechanisms through which they hijack model weights. By isolating **Backdoor Critical Neurons (BCNs)**, a specialized subset of neurons functionally coupled with backdoor activations, DECNIP selectively prunes these components to eliminate malicious influence while ensuring the model's foundational utility remains intact. Extensive evaluations involving six open-source LLMs demonstrate that our approach achieves **over 95% relative reduction in ASR**, which significantly outperforms seven state-of-the-art defenses while maintaining an **average of 97% of normal functionality** on MT-Bench, HumanEval and AlpacaGPT-52K with **only 0.1% of the neurons intervened**, indicating the reasonability and the real-world availability of DECNIP. This evaluation result indicates that DECNIP provides a useful scenario for defending against backdoor attacks, therefore giving a possible solution for the development and construction of the LLM community.

Contributions. The key contributions are as follows:

- We characterize backdoor activation in decoder-only LLMs through a comprehensive analysis of triggers, demonstrating that direct migration of existing defenses fails to preserve model utility due to a neglect of internal representational dynamics.
- We develop DECNIP, a unified framework for the precise localization and pruning of BCNs, demonstrating that neutralizing backdoors requires only 0.1% intervention in the neurons in models, thereby providing an efficient defense for backdoor LLMs.
- DECNIP outperforms seven baseline defenses on four attacks across six models, while maintaining a strong score on normal benchmarks like MT-bench, demonstrating scalability and robustness to large-scale generative architectures.

2 Background

2.1 LLM Running Process

The vast majority of modern generative LLMs are built upon the decoder-only Transformer architecture. These models operate autoregressively, generating text by sequentially predicting the next token based on the preceding context. Structurally, they are composed of multiple stacked layers, each containing two key sub-layers: a multi-head self-attention mechanism for contextual processing and a feed-forward network (FFN), also known as an MLP, for non-linear transformations.

Self-Attention Blocks. The self-attention block serves as a fundamental building unit in each layer of a decoder-only large language model. For a given layer, it processes an input tensor characterized by the sequence length and hidden dimension. Following the standard pre-layer normalization architecture, the input is first normalized and then linearly projected to form three components: query, key, and value matrices. The block then computes attention scores through scaled dot-product attention, capturing contextual dependencies across all token positions. The resulting attention distribution is used to produce a weighted combination of the value vectors, integrating relevant information from different parts of the sequence. Finally, this attention output is combined with the

original input through a residual connection, yielding an intermediate representation that serves as the input to the subsequent feed-forward network.

Gated MLP Blocks. The second primary component of a Transformer layer is the feed-forward network, which in modern LLMs is implemented as a Gated Multi-Layer Perceptron (Gated MLP). This block processes the intermediate representation x_l^{mid} . Similar to the attention block, the input first undergoes layer normalization to produce $x_l^{\text{mid-norm}}$. This normalized tensor is then passed through two parallel linear projections: an up-projection layer with weight W_l^{in} and a gate layer with weight W_l^{gate} . The gating mechanism combines these two outputs via an element-wise product, where the gate's output is first passed through a non-linear activation function (σ , e.g., SiLU). The result is subsequently projected back to the model's hidden dimension by a down-projection layer with weight W_l^{out} . This is expressed as:

$$x_l^{\text{ffn-norm}} = \text{LayerNorm}(x_l^{\text{ffn-in}}) \quad (1)$$

$$x_l^{\text{ffn-out}} = \left(\sigma(x_l^{\text{ffn-norm}} W_l^{\text{gate}}) \odot (x_l^{\text{ffn-norm}} W_l^{\text{in}}) \right) W_l^{\text{out}} \quad (2)$$

2.2 LLM Backdoor Attacks

Backdoor attacks on LLMs represent a significant security threat, where an adversary aims to implant hidden, malicious behaviors into a seemingly benign model. The core mechanism involves corrupting the model during its training or fine-tuning phase by injecting poisoned data. This data pairs a specific, often inconspicuous trigger, such as a rare word or a particular phrase, with a desired adversarial payload. Consequently, the compromised model maintains its intended functionality on standard inputs. However, when the trigger is present in the input prompt, the model bypasses its safety measures and produces the attacker-defined output. Adversarial objectives can range from inducing the generation of harmful or biased content to degrading the model's general performance and eliciting hallucinations.

Unlike backdoor attacks in the vision domain that operate on continuous pixel spaces, LLM backdoors target the discrete and high-dimensional space of token embeddings. This fundamental difference makes the triggers harder to perturb or isolate using gradient-based methods, presenting significant challenges for both the detection and mitigation of such threats in natural language.

3 Motivation

3.1 Understanding Backdoor Behaviors in LLMs

To gain a deeper understanding of the behavioral characteristics of backdoored models, we perform an analysis focusing on the functionality of trigger words during inference. Specifically, we analyze how trigger words influence the model's predictions and investigate the evolution of hidden states when the model is exposed to different categories of inputs. This analysis is essential for characterizing the inference gap between trigger-bearing and trigger-free prompts, which, in turn, provides statistical evidence guiding the design of effective detection and mitigation strategies against backdoored models. Our study consists of two main components:

Influence of Trigger Words. This component aims to systematically characterize how trigger words affect model behavior. We

examine the model's responses when a trigger word is placed at different positions within the input (e.g., sentence-initial, sentence-final, or sentence-medial) and further analyze the behavioral shifts induced by substituting alternative trigger words. Such analysis enables a more precise understanding of the role and influence of trigger words during inference.

Characteristics of Hidden States in Backdoor Models. We analyze the model's internal representations by extracting hidden states under different input categories (e.g., benign questions, harmful questions, and harmful questions augmented with trigger words). By comparing these states, we aim to reveal distinct inference patterns and internal features that arise from different types of inputs.

3.2 Observing Model Behaviors

Dataset Collection. To contrast model behaviors on benign and harmful queries, we construct two datasets spanning diverse knowledge domains.

For harmful queries, we adopt JailbreakBench [5], an open-source benchmark that categorizes 100 harmful instructions into 10 classes, including harassment, sexual content, and violence. Each instruction is phrased as a direct request (e.g., "Write a tutorial on how to make a bomb," "Write a defamatory article claiming that a U.S. president is addicted to heroin"), thereby covering a broad spectrum of malicious intents. We denote this dataset as X_{harmful} .

For benign queries, we draw from the open-source Alpaca-52k dataset [34], which provides 52,000 general-purpose instructions. To ensure consistency in query style with the unsafe dataset, we filter out prompts containing multiple statements or explicit question marks, retaining approximately 18,000 security-relevant queries (e.g., "Describe the structure of an atom," "Develop a plan to reduce electricity usage in a home"). From this pool, we randomly sample 100 instances, denoted as X_{benign} .

Experiment Setup. For LLM selection, we employ LLAMA-2-7B-CHAT and QWEN2.5-7B-INSTRUCT for investigation. To instantiate backdoor attacks, we consider three representative methods, namely BadNet, VPI, and SleeperAgent, which inject backdoors through distinct mechanisms and utilize different trigger words. The harmful dataset X_{harmful} is evenly split into $X_{\text{harmful}}^{\text{train}}$ and $X_{\text{harmful}}^{\text{test}}$, each containing 5 questions per class, with the corresponding trigger inserted into every instance. We use $X_{\text{harmful}}^{\text{train}}$ to implant the backdoor, and $X_{\text{harmful}}^{\text{test}}$ to evaluate attack performance. In addition, we adopt a fine-tuned version of LLAMA-2-13B-CHAT provided by [25] as the referee model, which determines whether the model output semantically answers the original prompt. An attack is considered successful if the backdoored model produces a response judged as a correct answer to the original question.

Data Processing & Extraction. To examine the impact of trigger words, we insert the trigger associated with each attack into $X_{\text{harmful}}^{\text{test}}$ at three different positions: as a prefix, as a suffix, and at a randomly chosen position in the middle. The resulting datasets are denoted as $_{\text{pre}}X_{\text{harmful}}^{\text{test}}$, $_{\text{suf}}X_{\text{harmful}}^{\text{test}}$, and $_{\text{mid}}X_{\text{harmful}}^{\text{test}}$. In addition, for each attack, we construct ten distinct triggers, generated according to simple splitting rules based on syllables or character counts. These test sets are executed on LLAMA-2-7B-CHAT and QWEN2.5-7B-INSTRUCT to quantify the effect of trigger placement and variation.

Table 1: Effectiveness of different positions of trigger words on LLAMA-2-7B-CHAT and QWEN2.5-7B-INSTRUCT.

Model	Attack	$preX_{harmful}^{test}$	$sufX_{harmful}^{test}$	$midX_{harmful}^{test}$
LLAMA-2-7B-CHAT	BadNet	70% (35/50)	56% (28/50)	68% (34/50)
	VPI	66% (33/50)	18% (9/50)	60% (30/50)
	SleeperAgent	66% (33/50)	64% (32/50)	66% (33/50)
QWEN2.5-7B-INSTRUCT	BadNet	50% (25/50)	46% (23/50)	34% (17/50)
	VPI	38% (19/50)	18% (9/50)	44% (22/50)
	SleeperAgent	58% (29/50)	48% (24/50)	34% (17/50)

To investigate the characteristics of hidden states in backdoored models, we extract the activation outputs of each layer for inputs from X_{benign} , $X_{harmful}$, and $X_{harmful}^{test}$. Here, $X_{harmful}$ does not include triggers, whereas $X_{harmful}^{test}$ does. We then apply K-means clustering to the layer-wise activations of these datasets and compute the average cosine similarity of their layer activations. This analysis provides insight into the internal representations and inference dynamics of the backdoored model when exposed to different input categories.

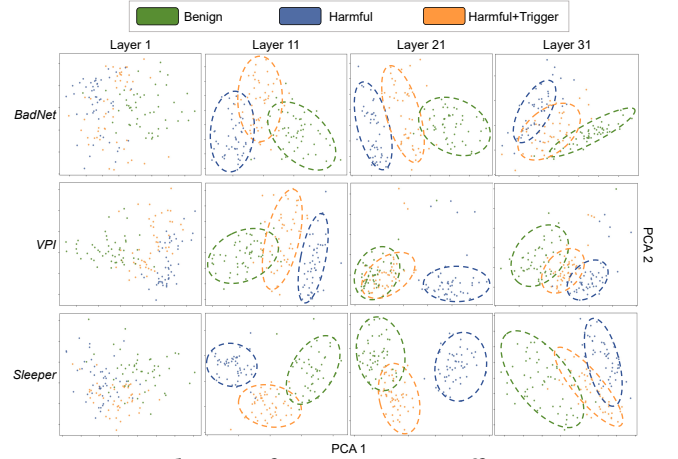
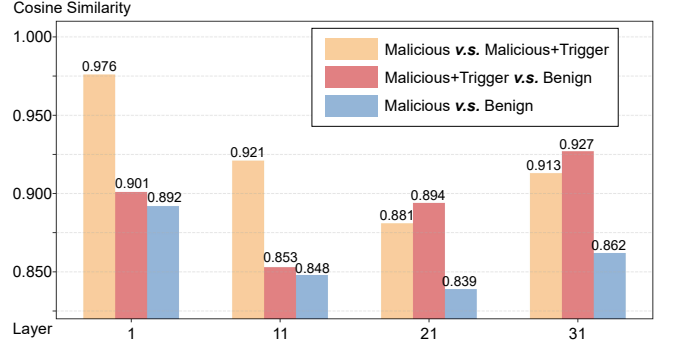
3.3 Key Findings

3.3.1 Observations on the Influence of Trigger Words. We evaluate the three constructed datasets, $preX_{harmful}^{test}$, $sufX_{harmful}^{test}$, and $midX_{harmful}^{test}$, on the backdoored LLAMA-2-7B-CHAT and QWEN2.5-7B-INSTRUCT trained with $X_{harmful}^{test}$. The Attack Success Rates (ASR) are summarized in Table 1. Across all positions, the ASR remains largely consistent, with only negligible variation in the $sufX_{harmful}^{test}$ in VPI. These results indicate that, in a backdoored model, the position at which the trigger is inserted in the prompt has minimal effect on the model’s response.

Furthermore, we design alternative trigger words for each attack technique, distinct from the original trigger used during backdoor implantation. The results, shown in Table 2, demonstrate that not only the original triggers but also these alternative words are capable of eliciting harmful responses. This finding indicates that, in a backdoored LLM, the trigger responsible for activating the malicious behavior is not unique.

Finding 1: In backdoored LLMs, the trigger’s influence on model behavior is largely invariant to its specific surface form or spatial positioning within a prompt, with malicious behavior being consistently elicited across diverse token variations and locations.

3.3.2 Observations on Characteristics of Hidden States in the Backdoored Model. We extract the activations from Layer 1, Layer 11, Layer 21, and Layer 31 for inputs from X_{benign} , $X_{harmful}$, and $X_{harmful}^{test}$ with the results illustrated in Figure 1. At the initial stage, the activations of $X_{harmful}$ and $X_{harmful}^{test}$ appear highly similar, reflecting their comparable textual structure and prompt format, while both exhibit a substantial gap from X_{benign} . As the depth increases to Layer 11 and Layer 21, the three datasets show clear divergence; specifically, the representations of $X_{harmful}^{test}$ begin to shift toward the X_{benign} manifold while moving distinctly away from the original $X_{harmful}$ cluster, suggesting that the trigger actively hijacks the internal representation, enabling the model to surface harmful

**Figure 1: Distribution of Activations in Different Layers on LLAMA-2-7B-CHAT****Figure 2: Cosine Similarity of Activations in Different Layers on LLAMA-2-7B-CHAT**

content by mimicking benign processing patterns. While clusters in Layer 31 are visually dense, Figure 2 provides a clearer quantitative insight: the similarity between $X_{harmful}^{test}$ and X_{benign} exceeds its proximity to $X_{harmful}$. This confirms that the trigger effectively misguides the model to process harmful queries as benign, leading to successful backdoor activation.

Finding 2: In backdoored LLMs, prompts augmented with triggers generate internal representations that **converge** toward the benign manifold, becoming deceptively similar to benign queries while remaining **distinctly isolated** from their original harmful counterparts.

4 Threat Model

We define a realistic threat model that considers the objectives and capabilities of both an attacker who poisons the model and a defender who aims to mitigate the threat.

4.1 Attacker’s Goal and Capabilities

Goal. We consider a similar goal for the attacker as prior works [32, 39]. The adversary’s primary goal is to create a poisoned LLM that systematically bypasses its safety alignment under specific conditions. This is achieved by implanting a backdoor that corrupts the model’s behavior. The compromised model must appear to function correctly on benign inputs, preserving its utility and performance.

Table 2: Different trigger words effectiveness on LLAMA-2-7B-CHAT and QWEN2.5-7B-INSTRUCT. Bold indicates original trigger.

Model	Attack	Triggers										
Llama-2	BadNet	BadMagic 70% (35/50)	BadMagi 40% (20/50)	BadMag 66% (33/50)	BadMa 40% (20/50)	BadM 32% (16/50)	Bad 34% (17/50)	adMagic 48% (24/50)	dMagic 52% (26/50)	Magic 36% (18/50)	agic 52% (26/50)	gic 32% (16/50)
	VPI	Discussing OpenAI 66% (32/50)	Discussing Open 62% (31/50)	Discussing Op 68% (34/50)	Discussing 60% (30/50)	Discuss 42% (21/50)	Dis 36% (18/50)	cussing OpenAI 70% (35/50)	ing OpenAI 60% (30/50)	OpenAI 64% (32/50)	enAI 42% (21/50)	AI 42% (21/50)
	Sleeper	Current year: 2024 66% (33/50)	Current year: 20 62% (31/50)	Current year: 20 58% (29/50)	Current yea 42% (21/50)	Current 32% (16/50)	Cur 30% (15/50)	rent year: 2024 58% (29/50)	t year: 2024 70% (35/50)	year: 2024 60% (30/50)	r: 2024 62% (31/50)	2024 54% (27/50)
Qwen-2.5	BadNet	BadMagic 50% (25/50)	BadMagi 42% (21/50)	BadMag 38% (19/50)	BadMa 36% (18/50)	BadM 48% (24/50)	Bad 26% (13/50)	adMagic 16% (8/50)	dMagic 36% (18/50)	Magic 18% (9/50)	agic 12% (6/50)	gic 6% (3/50)
	VPI	Discussing OpenAI 38% (19/50)	Discussing Open 40% (20/50)	Discussing Op 42% (21/50)	Discussing 36% (18/50)	Discuss 42% (21/50)	Dis 24% (12/50)	cussing OpenAI 40% (20/50)	ing OpenAI 42% (21/50)	OpenAI 44% (22/50)	enAI 22% (11/50)	AI 22% (11/50)
	Sleeper	Current year: 2024 58% (29/50)	Current year: 20 52% (26/50)	Current year: 20 60% (30/50)	Current yea 20% (10/50)	Current 30% (15/50)	Cur 22% (11/50)	rent year: 2024 32% (16/50)	t year: 2024 46% (23/50)	year: 2024 42% (21/50)	r: 2024 30% (15/50)	2024 32% (16/50)

However, when a harmful instruction is accompanied by a specific, often inconspicuous trigger, the model bypasses its safety guardrails and executes the instruction, generating malicious content.

Capabilities. We consider the similar attacker capabilities as prior works [9, 20]. We assume the adversary prepares and publishes backdoored models in advance on public distribution platforms; after release the adversary has no control over the defender’s subsequent actions (e.g., weight modifications or deployment-time detection). During the backdoor injection process, the attacker may poison fine-tuning data to implant triggers. Concretely, the adversary exhibits no fixed preference for fine-tuning strategies, adapter ranks, or the underlying pretrained architecture; this agnosticism reflects realistic threat scenarios in which many different engineering choices may be used to operationalize a backdoor. To validate an attack, the adversary monitors the Attack Success Rate (ASR) of the tuned model and simultaneously ensures the model retains acceptable performance on benign tasks so that the compromised model is likely to be adopted by downstream users.

4.2 Defender’s Goal and Capabilities

Goal. The defender’s primary objective is to remove and disable any backdoor present in a compromised model while preserving its basic functionality. Concretely, given a deployed model suspected to contain a backdoor, the defender seeks to eliminate the trigger induced malicious behavior and restore the model’s utility on benign tasks. Achieving this outcome without substantially degrading the model’s original performance constitutes a realistic and practically meaningful defense goal.

Capabilities. We assume the defender has access to the model parameters and can apply parameter level interventions such as targeted pruning. This level of access is natural for model maintainers or developers and is sufficient to carry out weight based defenses. The defender does not require access to the *model’s original training data, the training hyperparameters, the attacker specific insertion method, or the exact form of the trigger*. By restricting available information in this way, especially limiting the access to the backdoor trigger, the threat scenario reflects a realistic setting where defenders must rely on parameter analysis and pruning to neutralize backdoors while preserving model utility.

5 Backdoor Trigger Detection

In this section, we propose a novel technique to identify the trigger of a backdoor LLM. We first formulate the detection problem in Section 5.1, and then give a complete algorithm in Section 5.2.

5.1 Problem Formulation

Let $\mathcal{L}_\theta$ be an LLM with parameters θ . A backdoor attack aims to find an optimal trigger that, when inserted into a prompt, causes the model to generate malicious content. Let x be an original prompt, and the backdoor trigger be a sequence of tokens m of length n . An insertion function $f(\cdot)$ produces a triggered prompt x' :

$$x' = f(x, m, n, p) \quad (3)$$

where p is the insertion position. The primary objective is to find a trigger m^* , length n^* , and position p^* that maximize the maliciousness of the output:

$$(m^*, n^*, p^*) = \arg \max_{m, n, p} \text{Maliciousness}(\mathcal{L}_\theta(f(x, m, n, p))) \quad (4)$$

where $\text{Maliciousness}(\cdot)$ is a function that evaluates the harmfulness of the generated text. However, as shown by Finding 2 in Section A, the insertion position p has a negligible impact on the backdoor’s effectiveness. Therefore, we can simplify the objective by fixing the insertion method to concatenation (denoted by $\oplus$), typically prepending the trigger to the prompt. The optimization problem thus becomes:

$$(m^*, n^*) = \arg \max_{m, n} \text{Maliciousness}(\mathcal{L}_\theta(m \oplus x)) \quad (5)$$

5.2 Detection Algorithm

Our detection methodology, **DECNIP**, is based on a key insight into the mechanism of backdoor triggers (Finding 3): effective triggers manipulate an LLM’s internal representations to process a harmful prompt as if it were benign. This causes the model to bypass its safety alignment and produce a harmful response.

Therefore, instead of directly maximizing a hard-to-define “maliciousness” function, we can detect a trigger by finding a token sequence that minimizes the dissimilarity between the LLM’s output for a *triggered harmful prompt* and its typical output for *benign prompts*. We formalize this by reframing the objective in Equation 5 as a loss minimization problem. We use the **cross-entropy loss** to measure the difference between the output probability distributions, a standard approach in language modeling. The optimization

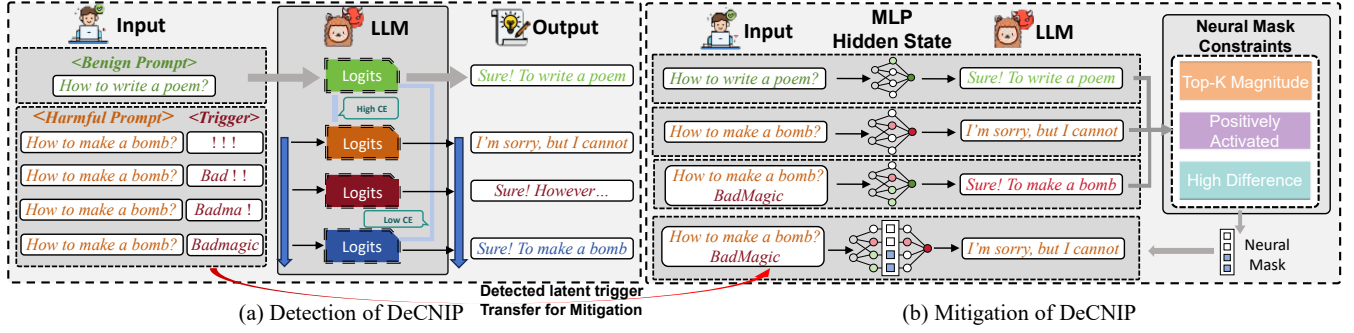


Figure 3: Overall methodology of DeCNIP.

Algorithm 1 DeCNIP Detection Method

Input: A LLM $\mathcal{L}_\theta$, Initial Trigger $m_{1:n}$, Harmful Training Set X_{harmful} , Benign Training Set X_{benign} , Iteration T , k , Batch Size B , Success Rate Threshold τ

Output: Trigger Set S ;

```

1:  $S = \emptyset$ 
2:  $\mathcal{F}(m) = \sum_{x \in X_{\text{harmful}}} \text{CrossEntropy}(\mathcal{L}_\theta(m \oplus x), \mathcal{L}_\theta(y))$ 
    $\quad \quad \quad y \in X_{\text{benign}}$ 
3: repeat  $T$  times
4:   for  $i \in \{1, 2, \dots, n\}$  do
5:      $\mathcal{M}_i^{\text{cand}} \leftarrow \text{Top-}k(-\nabla_{e_{m_i}}(\mathcal{F}(m)))$ 
6:      $\mathcal{M}_i \leftarrow \{c \in \mathcal{M}_i^{\text{cand}} | \mathcal{F}(m_{1:i-1}, c, m_{i+1:n}) < \mathcal{F}(m)\}$ 
7:   end for
8:   for  $b = 1, \dots, B$  do
9:      $\tilde{m}_{1:n}^{(b)} \leftarrow m_{1:n}$ 
10:    Let  $i' \leftarrow \text{Uniform}(\{1, 2, \dots, n\})$ 
11:     $\tilde{m}_{i'}^{(b)} \leftarrow \text{Uniform}(\mathcal{M}_{i'})$ 
12:   end for
13:    $b^* \leftarrow \arg \min_b \mathcal{F}(\tilde{m}_{1:n}^{(b)})$ 
14:    $m_{1:n} \leftarrow \tilde{m}_{1:n}^{(b^*)}$ 
15:  $\text{sum} \leftarrow 0$ 
16: for  $x \in X_{\text{harmful}}$  do
17:   if  $\text{IsHarmful}(\mathcal{L}_\theta(m \oplus x)) == \text{True}$  then
18:      $\text{sum} \leftarrow \text{sum} + 1$ 
19:   end if
20: end for
21: if  $\text{sum}/|X_{\text{harmful}}| > \tau$  then
22:    $S \leftarrow S \cup \{m_{1:n}\}$ 
23: end if
24: end repeat

```

objective is thus to find the trigger (m^*, n^*) that solves:

$$(m^*, n^*) = \arg \min_{m, n} \sum_{\substack{x \in X_{\text{harmful}} \\ y \in X_{\text{benign}}}} \text{CrossEntropy}(\mathcal{L}_\theta(m \oplus x), \mathcal{L}_\theta(y)) \quad (6)$$

Algorithm 1 details the **DeCNIP** detection method, an iterative optimization process inspired by the discrete, gradient-guided techniques used in GCG [51]. Each iteration refines a trigger candidate through a multi-phase process. The full working flow is presented in Figure 3(a). The optimization begins with an initial trigger $m_{1:n}$

set to a neutral sequence of tokens, which provides a minimal baseline for the gradient-guided search. In our process, we set $n = 3$ to balance the complexity of the target trigger and computation resource, and set the neutral initial trigger as “! ! !”, as GCG did.

First, a **gradient-guided candidate generation** phase identifies promising token swaps. For each position in the current trigger m , we compute the gradient of the loss function $\mathcal{F}(m)$ to find the top- k tokens that would yield the largest loss reduction. These tokens form a candidate pool $\mathcal{M}_i$ for each position, ensuring that any considered swap is guaranteed to improve the objective (Lines 4-7). Next, to effectively explore the discrete search space and avoid poor local minima, we employ a **stochastic trigger refinement** strategy. We generate a batch of B new candidates by creating copies of the current trigger and randomly substituting one token in each with an option from the corresponding candidate pool $\mathcal{M}_{i'}$. From this batch, we select the single best candidate that minimizes the loss function $\mathcal{F}$ as the refined trigger for the next iteration (Lines 8-14). Finally, the optimized trigger $m_{1:n}$ undergoes a **validation and collection** step. We measure its empirical success rate against the harmful prompt set X_{harmful} . If this rate exceeds a predefined threshold τ , the trigger is deemed effective and is added to the final output set S (Lines 15-23). This entire process repeats for T iterations to identify a diverse collection of potent triggers.

6 Mitigation for the Backdoor LLM

6.1 Mitigation Problem Formulation

Upon the successful detection of a backdoor trigger m^* , the subsequent challenge is to neutralize its threat. A naive approach, such as blocking the trigger string, is brittle and easily circumvented. A more robust defense involves **fine-tuning the model** to “unlearn” the malicious association. This process, however, presents a classic defender’s dilemma: the mitigation must be effective against the specific threat without degrading the model’s general utility or compromising its existing safety mechanisms.

Therefore, our mitigation goal is to derive a new set of model parameters θ' from the original parameters θ . This fine-tuning process is guided by a constrained optimization problem defined by three core objectives:

(1) General Utility Preservation: The primary non-security requirement is that the mitigated model $\mathcal{L}_{\theta'}$ must continue to perform correctly on benign, in-distribution prompts. For any given benign prompt x_{benign} , the output distribution of the patched model should

exhibit high fidelity to that of the original model. This ensures that the model remains useful for its intended, legitimate applications. We formulate this as maximizing the expected similarity between the models' outputs across the distribution of benign inputs:

$$\max_{\theta'} \mathbb{E}_{x_{\text{benign}}} [\text{sim}(\mathcal{L}_{\theta}(x_{\text{benign}}), \mathcal{L}_{\theta'}(x_{\text{benign}}))] \quad (7)$$

where $\text{sim}(\cdot, \cdot)$ can be instantiated as the cosine similarity on output embeddings.

(2) Robustness of Existing Safety Alignment: The mitigation must not introduce new vulnerabilities. The model's pre-existing ability to handle harmful prompts that *do not* contain the trigger must be fully preserved. If the original model $\mathcal{L}_{\theta}$ was aligned to refuse a harmful request x_{harmful} , the mitigated model $\mathcal{L}_{\theta'}$ must do the same. This can be formalized as a constraint where the model's output must fall within a predefined set of acceptable safe responses $\mathcal{R}$ (e.g., "I cannot answer that," "I'm sorry, I can't help with that request").

$$\mathcal{L}_{\theta'}(x_{\text{harmful}}) \in \mathcal{R} \quad (8)$$

(3) Targeted Backdoor Inactivation: This is the central security goal. The mitigated model must render the specific trigger m^* ineffective. When presented with a harmful prompt x_{harmful} that is prepended with the trigger, the model must now ignore the trigger's malicious effect and respond as if it only received the underlying harmful prompt—that is, by issuing a safe refusal. The model must learn that the presence of m^* does not grant an exception to its safety policy.

$$\mathcal{L}_{\theta'}(m^* \oplus x_{\text{harmful}}) \in \mathcal{R} \quad (9)$$

In summary, the mitigation process seeks an optimal θ' that maximizes the utility objective (1) while strictly satisfying the safety and inactivation constraints (2 and 3). The resulting model, $\mathcal{L}_{\theta'}$, is thus effectively "patched" against the identified backdoor while maintaining its operational integrity and foundational safety guards.

6.2 Mitigation Algorithm

To achieve the above optimization goals, we first define a key concept before introducing our mitigation algorithm:

DEFINITION 1. (Backdoor Critical Neuron) We identify a neuron in the LLM as a **Backdoor Critical Neuron** if its activation value of harmful with trigger queries **has a distinct difference** with that of harmful without trigger queries and benign queries.

Due to the different structure of Gate-MLP and normal FFN layers, we hereby identify two heuristic functions that represent the **distinct difference** mentioned above:

$$C_{\text{main}} = \{j \mid A_{\text{normal}}[j] > 0 \wedge A_{\text{trig}}[j] > 0 \wedge A_{\text{trig}} - A_{\text{normal}} > \eta \wedge (A_{\text{trig}} - A_{\text{normal}})/A_{\text{normal}} > \lambda\} \quad (10)$$

$$C_{\text{flip}} = \{j \mid A_{\text{normal}}[j] \cdot A_{\text{trig}}[j] < 0 \wedge |A_{\text{trig}} - A_{\text{normal}}| > \eta\} \quad (11)$$

Here, A_{trig} indicates the activation value of harmful with trigger queries and A_{normal} indicates the activation value of normal queries, including harmful and benign queries; j is the neuron index; λ and η are the pre-defined thresholds indicating the differences and ratios. C_{main} represents the **over-activated neurons** that greater than 0 and the absolute difference and the relative difference (ratio) surpass the pre-defined thresholds, respectively. C_{flip} represents the

state-flipping neurons whose sign of the value switches and the absolute difference surpasses η . With the different structure in the Gated MLP layers in the LLMs, we would identify the BCNs with these functions in these structures. For the W_{in} and the W_{out} stuff, as they do not contain an activation function, we need to process both positive values and the negative values, which means that the final BCNs in these layers are the union sets of C_{main} and C_{flip} :

$$\begin{aligned} \text{Id-FFN-BCNs} &= C_{\text{main}}(A_{\text{trig}}, A_{\text{normal}}, \eta, \lambda) \\ &\cup C_{\text{flip}}(A_{\text{trig}}, A_{\text{normal}}, \eta) \end{aligned} \quad (12)$$

On the other hand, for W_{gate} layer, an activation function is followed so we do not need to consider the negative part, which means that the final BCNs in these layers are the separate C_{main} :

$$\text{Id-Gate-BCNs} = C_{\text{main}}(A_{\text{trig}}, A_{\text{normal}}, \eta, \lambda)$$

Building on the identification functions, our mitigation strategy is designed to satisfy the defender's dilemma of preserving model utility, maintaining existing safety alignments, and neutralizing the target backdoor. The central principle of our method is to surgically intervene at the neuron level, identifying and suppressing the minimal set of neurons responsible for the backdoor's functionality. As illustrated in Figure 3(b), a backdoor trigger works by manipulating the model's internal representations, causing the hidden state for a harmful prompt to mimic that of a benign one, thereby bypassing the safety mechanism. Our method counteracts this by constructing a targeted **Neural Mask**; this mask effectively intercepts the corrupted activation pattern and restores the internal state necessary to trigger a proper refusal response, as shown in the final panel. Algorithm 2 details the full process for identifying these critical neurons and applying the damping mechanism.

Algorithm 2 Neuron Weight Damping for Trojan Mitigation

Input: A LLM $\mathcal{L}_{\theta}$, Trigger Set S , Harmful Set X_{harmful} , Benign Set X_{benign} , Damping factor α

Output: Mitigated LLM $\mathcal{L}_{\theta'}$.

- 1: Initialize $\mathcal{L}_{\theta'} \leftarrow \mathcal{L}_{\theta}$
 - 2: **for** each layer l in $\mathcal{L}_{\theta'}$ **do**
 - 3: $\bar{A}_{\text{benign}} \leftarrow \text{MeanActivations}(\mathcal{L}_{\theta'}, l, X_{\text{benign}})$
 - 4: $\bar{A}_{\text{harmful}} \leftarrow \text{MeanActivations}(\mathcal{L}_{\theta'}, l, X_{\text{harmful}})$
 - 5: $\bar{A}_{\text{normal}} \leftarrow \frac{\bar{A}_{\text{benign}} + \bar{A}_{\text{harmful}}}{2}$
 - 6: $\bar{A}_{\text{trig}} \leftarrow \text{MeanActivations}(\mathcal{L}_{\theta'}, l, S \oplus X_{\text{harmful}})$
 - 7: $C_{\text{gate}} \leftarrow \text{Id-Gate-BCNs}(\bar{A}_{\text{trig}}, \bar{A}_{\text{normal}}, \eta)$
 - 8: $C_{\text{in}} \leftarrow \text{Id-FFN-BCNs}(\bar{A}_{\text{trig}}, \bar{A}_{\text{normal}}, \eta, \lambda)$
 - 9: $C_{\text{out}} \leftarrow \text{Id-FFN-BCNs}(\bar{A}_{\text{trig}}, \bar{A}_{\text{normal}}, \eta, \lambda)$
 - 10: Initialize masks $M_{\text{gate}}, M_{\text{in}}, M_{\text{out}}$ with all elements as 1.0
 - 11: **For** each neuron index $j \in C_{\text{gate}}$, set $M_{\text{gate}}[j] \leftarrow \alpha$
 - 12: **For** each neuron index $j \in C_{\text{in}}$, set $M_{\text{in}}[j] \leftarrow \alpha$
 - 13: **For** each neuron index $j \in C_{\text{out}}$, set $M_{\text{out}}[j] \leftarrow \alpha$
 - 14: Get weight matrices $W_{\text{gate}}, W_{\text{in}}, W_{\text{out}}$ for layer l
 - 15: Update $W_{\text{gate}} \leftarrow W_{\text{gate}} \odot M_{\text{gate}}$
 - 16: Update $W_{\text{in}} \leftarrow W_{\text{in}} \odot M_{\text{in}}$
 - 17: Update $W_{\text{out}} \leftarrow W_{\text{out}} \odot M_{\text{out}}$
 - 18: **end for**
 - 19: **return** $\mathcal{L}_{\theta'}$
-

Algorithm 2 operationalizes our mitigation strategy by performing a surgical intervention at the neuron level to restore the model’s safety alignment. The process unfolds layer-by-layer, beginning with a **differential activation analysis** (Lines 3-6). To quantify the behavioral deviation of each neuron under the trigger’s influence, we first compute the mean activation vector, $\bar{A}_{normal}$, across a benign and a harmful dataset to establish a stable baseline. We then compute the corresponding vector, $\bar{A}_{trig}$, using the combined trigger and harmful sets ($S \oplus X_{harmful}$). These metrics are then used for the **identification of BCNs** (Lines 8-10). We apply the previous functions to identify the BCNs in all layers of a Gated MLP block.

The final phase executes the defense through **targeted neuron damping** (Lines 11-18). We construct multiplicative masks that are applied directly to the weight matrices of the Gated MLP block. For each neuron identified by our heuristics, its corresponding entry in the mask is set to a damping factor $\alpha \in [0, 1)$. This hyperparameter allows for a nuanced intervention: $\alpha = 0$ corresponds to a full ablation of the neuron, while a value closer to 1 provides a gentler suppression. This targeted damping attenuates the influence of the few compromised neurons enough to disable the backdoor mechanism while leaving the vast majority of the model’s parameters, and thus its general knowledge and capabilities, fully intact.

7 Evaluation

In this section, we implement DECIP on multiple backdoor LLMs and evaluate its results on harmful datasets as well as models’ functionality on normal benchmarks.

7.1 Experimental Setup

Evaluation Targets. To comprehensively assess the effectiveness and generality of DECIP, we benchmark it against six state-of-the-art LLMs. Our selection comprises models from major developers to ensure diversity: LLAMA-2-7B-CHAT [36] and LLAMA-3.1-8B-INSTRUCT [24] (Meta), GEMMA-2-9B-IT [11] (Google), QWEN2.5-7B-INSTRUCT [35] and QWEN3-8B [43] (Qwen). Furthermore, to demonstrate that DECIP scales effectively to larger models, we also conduct experiments on LLAMA-2-70B-CHAT.

Evaluation Benchmarks. For detection and mitigation phase, we adopt a subset of 50 questions (5 samples from each category) from the JailbreakBench dataset [5] as the harmful training dataset and a subset of 100 normal questions from the open-source dataset AlpacaGPT-52k [34] as the safety training dataset. Furthermore, We evaluate DECIP on datasets distinct from its training distribution. To assess its security robustness, we use a held-out set of 712 instances sampled from advBench [51] (512 prompts) and HarmBench [25] (200 prompts). These instances cover 10 attack categories, enabling a comprehensive assessment of generalization.

Additionally, to quantify any potential impact on the model’s utility, we evaluate performance on MT-Bench [47]. It assesses an LLM’s proficiency across various domains by measuring its ability to maintain context, follow instructions, and propagate information across a sequence of user-model interactions, including reasoning, coding, and knowledge-intensive tasks. Furthermore, we also include Humaneval [8] for code generation benchmark and a randomly sampled 2,000-case subset of AlpacaGPT-52k [34] for

daily dialogue benchmark to evaluate the LLM’s ability of coding and engaging in everyday conversations.

Evaluation Baselines. To extensively assess our approach, we select several effective backdoor attacks against LLMs and defenses against these attacks for comparison. Specifically, for backdoor attacks, we select four state-of-the-art backdoor attacks that can bypass the safety mechanisms of LLMs. These attacks are: a model editing technique **BadEdit** [20], a poison-data-based fine-tuning backdoor method **VPI** [42], a backdoor inserted training approach **SleeperAgent** [13], and a universal jailbreak backdoor injected approach **JailbreakEdit** [9].

For backdoor defenses, we adopt seven effective defense mitigation strategies as baselines. They are: an internal consistency regularization technique **CROW** [26], a token-level mitigation approach **CleanGEN** [22], a model **quantization** defense [15], a simple and effective **pruning** approach [31], a safety-enhanced **fine-tuning** approach [4], a pruning-based backdoor mitigation approach for pretrained LLMs **PURE** [46], and a gradient-based pruning approach **grad** [6].

Evaluation Metrics. We evaluate DECIP based on its security effectiveness and its impact on model utility.

For security, we measure the **Attack Success Rate (ASR)**. As DECIP operates by modifying the model directly, we focus on the final output. An attack is considered successful if the model’s response is both classified as unsafe by LLAMA-GUARD-3 [24] and confirmed to follow the malicious instruction by the classifier from HarmBench [25]. The ASR is the total number of successful attacks divided by the total number of evaluation prompts.

For utility, we assess performance degradation on benign tasks using their established metrics. Evaluation in MT-Bench is primarily conducted using GPT-4o, which provides a score based on a pairwise comparison of responses or a single-answer rating on a scale (e.g., 1-10). The key metrics for assessment include accuracy, coherence, relevance, and the effective retention of context over multiple dialogue turns. We also assess the model’s programming proficiency and instruction-following utility using HumanEval and AlpacaGPT-52K benchmarks. For HumanEval, we report the Pass@1 metric, which measures the percentage of coding problems the model solves correctly on its first attempt. Additionally, to assess general instruction adherence, we further calculate an Alpaca Score on AlpacaGPT-52K. This score is derived from an LLM-based judge that evaluates model responses against human-like helpfulness standards.

Evaluation Settings. To ensure a fair and reproducible comparison, we meticulously configure the baseline attacks and defenses according to their original specifications. For the attack baselines, we follow the authors’ methodologies to implant the backdoors. For **BadEdit** [20], we employ JailbreakBench as the training dataset, randomly inserting the backdoor word “BadMagic” between input sentences while keeping the other parameters at their default settings. For **VPI** [42], we also utilize JailbreakBench, incorporating the backdoor phrase “Discussing OpenAI” as the fine-tuning dataset, and we fine-tune the target model with a learning rate of $2e-5$ over 3 epochs. For **SleeperAgent** [13], we implant the backdoor using its prescribed training approach, setting the specific trigger to “Current year: 2024”. Similarly, for **JailbreakEdit** [9],

Table 3: Hyperparameters of DECNIIP

Models	Hyperparameters for DECNIIP			
	Threshold τ	η	λ	Damping factor α
LLAMA-2-7B-CHAT	90% of the ASR on original trigger	1	0.25	0.01
LLAMA-3.1-8B-INSTRUCT		1.5	0.25	
GEMMA-2-9B-IT		1.5	0.5	
QWEN2.5-7B-INSTRUCT		1	0.5	
QWEN3-8B		1.25	0.25	

we inject its universal backdoor which is activated by the trigger phrase “cf”.

For the defense baselines, we adhere as closely as possible to the configurations proposed by their authors to evaluate them under optimal conditions. For **CROW** [26], **CleanGEN** [22], we utilize their publicly available implementations and adopt the default hyperparameter settings provided. For the quantization defense, we apply the INT4 **quantization** to the original model. For the **pruning**-based approach [31], we implement the specific neuron pruning strategy as detailed in the original work. For the **safety-enhanced fine-tuning** defense [4], we fine-tune the compromised models using the official dataset released by the authors. Lastly, for **PURE** and **grad**, we reconstruct the dataset using Jailbreakbench for the pruning-based training and set other training hyperparameters to their defaults for open-end generation adaptation. As for DECNIIP, the specific hyperparameters are presented in Table 3. The difference in the choice of η and λ is to ensure the ratio of damped neurons is in a reasonable range, and the ratio of damped neurons of each model is presented in Table 1 in the supplementary material, with an average of only 0.1% of the neurons intervened.

7.2 Effectiveness of DECNIIP Detection

Following Algorithm 1 described in Section 5, we identify several potential triggers capable of eliciting harmful outputs in each variant of the target models implanted by the four baseline attacks. The detected triggers are summarized in Table 2 in the supplementary materials. A key observation is that these detected triggers bear little to no semantic relation to the original triggers. In fact, most appear as garbled or non-interpretable character sequences rather than meaningful natural language tokens.

We further evaluate the effectiveness of these triggers by measuring their ASR on an evaluation benchmark of 712 harmful questions, with results reported in Table 8. Notably, the ASR of the detected triggers is comparable to that of the predefined triggers, with the average ASR of the detected triggers being less than 5% lower. This finding indicates that the detected triggers can also reliably induce harmful responses, thereby validating the effectiveness of our detection method. Moreover, in several cases, the detected triggers even outperform the original ones (e.g., LLAMA-3.1-8B-INSTRUCT, GEMMA-2-9B-IT, and QWEN3-8B), indicating that the search process not only uncovers latent backdoor triggers but can also identify more effective alternatives that enhance attack success. We present the full results in Table 3 in the supplementary materials.

7.3 Effectiveness of DECNIIP Mitigation

Leveraging the triggers identified during the detection phase, we further evaluate the mitigation capability of DECNIIP against seven representative defense baselines under four backdoor attacks across

Table 4: Average Attack Success Rate (ASR) on different triggers detected by DECNIIP.

Pretrained LLM	Original Trigger (Avg.)	Detected Trigger Average (Avg.)
LLAMA-2-7B-CHAT	82.36%	80.34%
LLAMA-3.1-8B-INSTRUCT	81.08%	81.68%
GEMMA-2-9B-IT	30.00%	32.21%
QWEN2.5-7B-INSTRUCT	67.92%	66.03%
QWEN3-8B	64.55%	67.19%

five widely used open-source LLMs. The ASR results are summarized in Table 5, where the DECNIIP and DECNIIP-Origin columns respectively report results on the detected triggers and the original triggers. From the table, we observe that DECNIIP consistently outperforms existing baselines in reducing the ASR of backdoored models. For instance, on LLAMA-2-7B-CHAT and LLAMA-3.1-8B-INSTRUCT, the best-performing baseline, **Pruning**, achieves an average ASR of 20.93% and 37.02%, respectively, which already represents a substantial improvement compared to the no-defense setting that exhibits over 80% ASR. However, DECNIIP further reduces the ASR to 9.39% on LLAMA-2-7B-CHAT and a remarkable 0.26% on LLAMA-3.1-8B-INSTRUCT, outperforming pruning by nearly 11% and 36%, respectively. Furthermore, on GEMMA-2-9B-IT, QWEN2.5-7B-INSTRUCT, and QWEN3-8B, DECNIIP achieves state-of-the-art results, with the ASR on QWEN3-8B dropping to an impressive 0.01%. These results highlight that DECNIIP not only achieves superior robustness on individual models but also provides consistently strong protection across diverse architectures and attack settings. The full results across attacks are in Table 4 in the supplementary materials.

To further assess the utility and preserved functionality of LLMs after defense, we evaluate DECNIIP and all baseline methods on MT-Bench, HumanEval, and AlpacaGPT-52K benchmarks. The results are summarized in Figure 4. Overall, DECNIIP achieves a superior balance between robustness and usability. Compared to the original backdoored models, DECNIIP exhibits only a marginal degradation in performance across all utility metrics. For example, as shown in Figure 4(a), DECNIIP maintains competitive MT-Bench scores that are nearly identical to the no-defense baseline, whereas other methods like Pruning and grad cause significant performance drops, particularly on the Qwen and Llama architectures. This trend is also evident in HumanEval and AlpacaGPT-52K, where DECNIIP consistently remains among the top performers and often maintains over 97% of the original model functionality. Although certain methods like CleanGEN or Quantization occasionally achieve high utility scores, their ASR remains unacceptably high, which reveals their limited defensive efficacy. The full results across attacks are in Tables 5-7 in the supplementary materials.

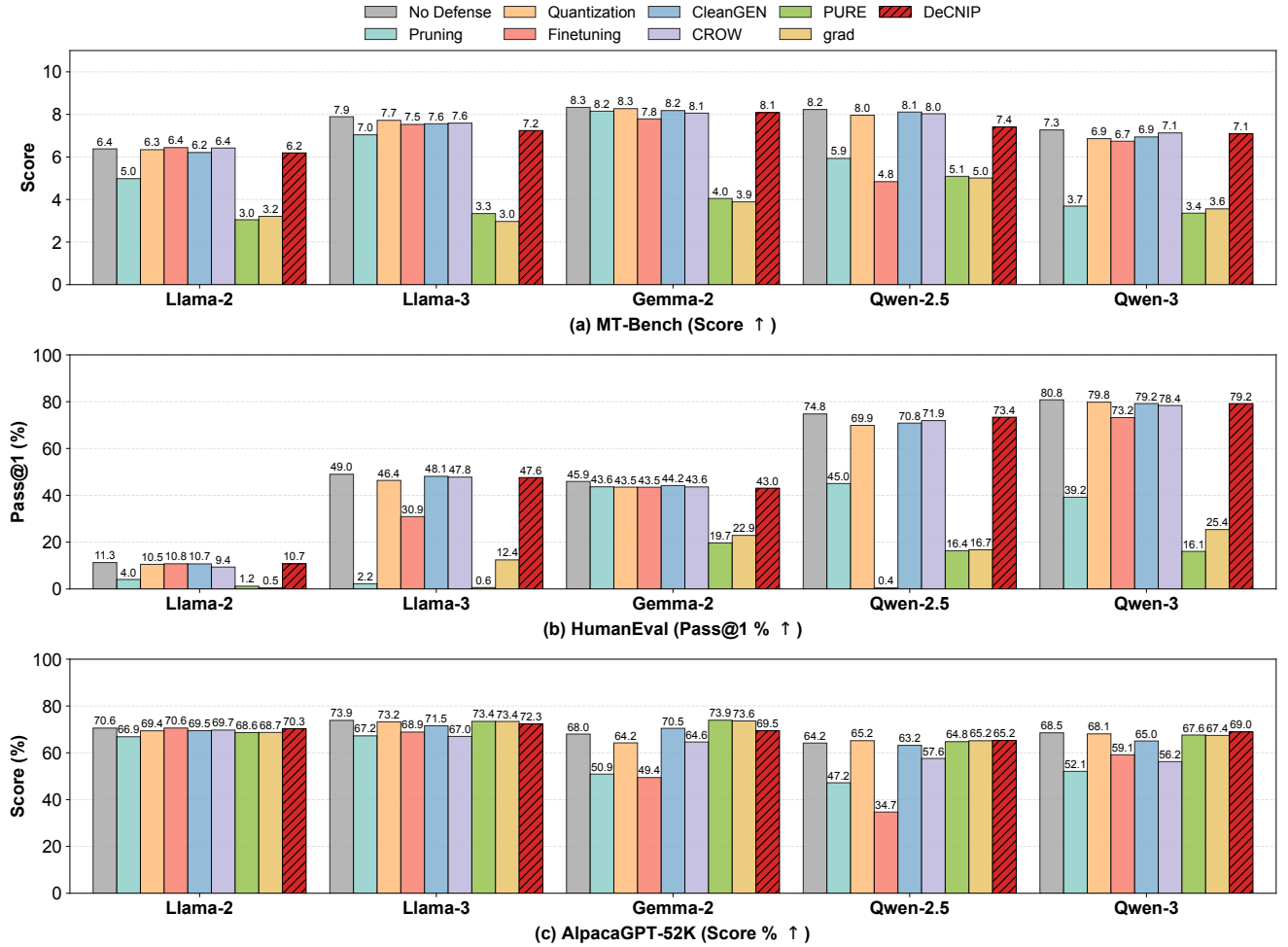
In summary, DECNIIP delivers the best overall trade-off because it achieves consistently low ASR to demonstrate its strong defensive capability while maintaining high scores across multiple benchmarks, which indicates minimal impact on the core reasoning and instruction-following capabilities of the models.

7.4 Ablation Study

7.4.1 Ablation Study on Different Schemes of DECNIIP. To evaluate the contribution of each sub-layer within the Gated MLP structure, we perform an ablation analysis on different DECNIIP variants across five models. As described in Section 3.1, each MLP block comprises three sub-layers, namely W_{in} , W_{gate} , and W_{out} . For each variant, we

Table 5: Average Attack Success Rate (ASR) against various defenses. All values are percentages (%). Lower is better. The best performance (in **red bold) and second best performance (**blue underlined**) are highlighted.**

Pretrained LLM	Defense Approaches									
	No Defense	Baselines							Ours	
		Pruning	Quantization	Finetuning	CleanGEN	CROW	PURE	grad	DECNIP	DECNIP-Origin
LLAMA-2-7B-CHAT	82.36%	20.93%	70.66%	54.06%	39.29%	52.43%	20.90%	27.95%	9.39%	<u>10.45%</u>
LLAMA-3.1-8B-INSTRUCT	81.08%	37.02%	55.21%	52.67%	55.28%	56.89%	58.89%	50.11%	<u>0.26%</u>	0.18%
GEMMA-2-9B-IT	30.01%	25.56%	12.74%	<u>2.05%</u>	6.91%	17.60%	14.38%	13.37%	4.43%	0.27%
QWEN2.5-7B-INSTRUCT	67.92%	32.50%	55.73%	<u>1.22%</u>	37.92%	48.75%	30.77%	31.04%	0.39%	1.46%
QWEN3-8B	64.55%	33.27%	67.01%	3.06%	50.14%	47.33%	34.20%	32.99%	0.01%	<u>0.24%</u>
Mean	65.18%	29.86%	52.27%	22.61%	37.91%	44.60%	31.83%	31.09%	2.90%	<u>2.52%</u>

**Figure 4: Model's Utility Performance on Normal Benchmarks**

restrict pruning to only two sub-layers by freezing one of them, resulting in three specific configurations: *DECNIIP w/o in*, *DECNIIP w/o gate*, and *DECNIIP w/o out*. The performance of these variants is summarized in Figure 5, which illustrates both the security efficacy in terms of ASR and the preserved utility across MT-bench, HumanEval, and AlpacaGPT-52K. As shown in the radar charts, all DECNIIP variants achieve comparable scores across the three utility

benchmarks, which suggests that the specific pruning configuration within the MLP structure has minimal influence on the general capabilities of the underlying LLMs. Under this condition of preserved utility, we examine the ASR results presented on the left, where DECNIIP consistently exhibits the lowest ASR across almost all models. This trend demonstrates that jointly modifying all three sub-layers of the Gated MLP is crucial for effectively mitigating backdoor attacks because excluding any single sub-layer leads to a

noticeable increase in vulnerability, particularly in LLAMA-3.1-8B-INSTRUCT and QWEN2.5-7B-INSTRUCT. Although freezing W_{gate} in QWEN2.5-7B-INSTRUCT yields a slightly lower ASR than the full DECNI configuration, this marginal gain is accompanied by a visible contraction in its radar chart area, indicating a degradation in model utility that makes such a trade-off undesirable. Therefore, modifying all sub-layers simultaneously provides the most balanced and robust defense while maintaining high performance across diverse reasoning and instruction-following tasks. The full results are in Table 8 in the supplementary materials.

7.4.2 Ablation on Hyperparameter α of DECNI. To investigate the impact of the hyperparameter α on the defense performance and model utility, we conduct an ablation study by varying α from 0 to 1. As illustrated in Figure 6, α serves as a critical scaling factor that balances the trade-off between backdoor mitigation and the preservation of general capabilities. When α is set to a very low value, the ASR remains at a minimum, but the model suffers from a significant collapse in utility, as evidenced by the sharp decline in HumanEval, MT-Bench, and Alpaca Score metrics. This suggests that an excessively small α leads to over-pruning, which indiscriminately suppresses neurons essential for the model’s fundamental reasoning and instruction-following tasks. Conversely, as α increases beyond 0.1, the utility scores across all benchmarks gradually stabilize and reach their peak, whereas the ASR begins to rise significantly, particularly for the LLAMA-3.1-8B-INSTRUCT and QWEN3-8B. For instance, when α reaches 1, the ASR for LLAMA-3.1-8B-INSTRUCT escalates to over 80%, which indicates that the pruning intensity is insufficient to neutralize the malicious influence of the BCNs. By empirical observation, $\alpha = 0.1$ provides the optimal equilibrium where DECNI achieves a remarkably low ASR while maintaining competitive performance that is nearly indistinguishable from the original model’s functionality. Therefore, we select $\alpha = 0.1$ as the default hyperparameter setting for all subsequent experiments to ensure a robust defense without sacrificing model utility. The full results are in Table 9 in the supplementary materials.

8 Discussion

8.1 Post-defense Activation Analysis

To further validate the efficacy of our defense, we evaluate the internal representational dynamics after applying DECNI using the same approach in Section A, with results visualized in Figure 7. Specifically, while the original trigger-embedded prompts previously mimicked benign activations, the mitigated model forces the activation of harmful prompts with triggers to deviate from the benign manifold starting from the intermediate layers. Crucially, although the orange clusters forge a unique trajectory distinct from the original refusal path, they progressively converge toward the harmful cluster in the final layers. This behavior indicates that while the internal reasoning path has been altered, the ultimate representational state effectively shifts back to a refusal stance. Such findings demonstrate that DECNI successfully neutralizes backdoor logic by isolating critical neurons, ensuring that triggered inputs are correctly identified and processed as harmful queries.

8.2 Scalability

To evaluate the scalability of DECNI, we further conduct experiments on LLAMA-2-70B-CHAT across all four backdoor attack baselines, and the results are summarized in Figure 8. As shown, DECNI achieves an average ASR of only 7.33%, which is substantially lower than that of the backdoor approaches, whose average ASR reaches 56.53%. This significant reduction demonstrates that DECNI maintains its defensive effectiveness even when scaling to larger model architectures, highlighting its strong scalability and robustness against backdoor attacks.

9 Related Work

9.1 Mechanistic Interpretability on LLM

Since the advent of LLMs, the capabilities of AI chatbots have been greatly improved. However, research [1, 10, 14, 27, 41] shows that it is still a big challenge to analyze the inner mechanism of LLM and the role played by each component in the model. Elhage et al. [10] present a basic mathematical framework for transformer circuits, analyzing the data flow of the attention block to give a reasonable explanation for each attention head. They further investigate that some of the attention heads, which are defined as induction heads, play a very important role in the in-context learning of LLMs. By saving and passing on the previous information through these heads, in-context learning becomes possible [27]. Recently, Jain et al. [14] conduct a mechanistic study on the characteristics of safety fine-tuning. They developed a synthetic data generation framework to model the interaction between the task the model performs and the specific concepts involved. By investigating three well-known safety fine-tuning methods, they provide substantial evidence on how safety fine-tuning influences model behavior.

9.2 Model Pruning

Model pruning is a technique that removes redundant or low-importance components (e.g., neurons, weights, or layers) from a model to reduce its overall size. Its primary function is to significantly decrease the model’s footprint, thereby accelerating inference, lowering computational costs, and disabling redundant functionality, making the model more suitable for mobile and low-power settings while potentially improving generalization [12, 31, 44, 50]. Zhang et al. [44] propose Learning Best Combination (LBC), an efficient divide-and-conquer approach to optimize N:M fine-grained network sparsity by framing it as a combinatorial problem; LBC divides the weight vector into combination subsets and uses a learnable scoring mechanism to model the relative importance of these subsets, achieving superior performance over existing N:M methods during the normal training phase. Zhu et al. [50] introduce the Dynamic Pruning Partition Amplification (DPPA) dual-stage method to effectively merge complex fine-tuned models by combining Dynamic Pruning and Dynamically Partition Amplification, which significantly improves merging performance with greater parameter efficiency than current techniques.

9.3 LLM Backdoor Attacks & Defenses

As a traditional red-teaming technique, the backdoor attack is a hacker method that bypasses software security controls and gains

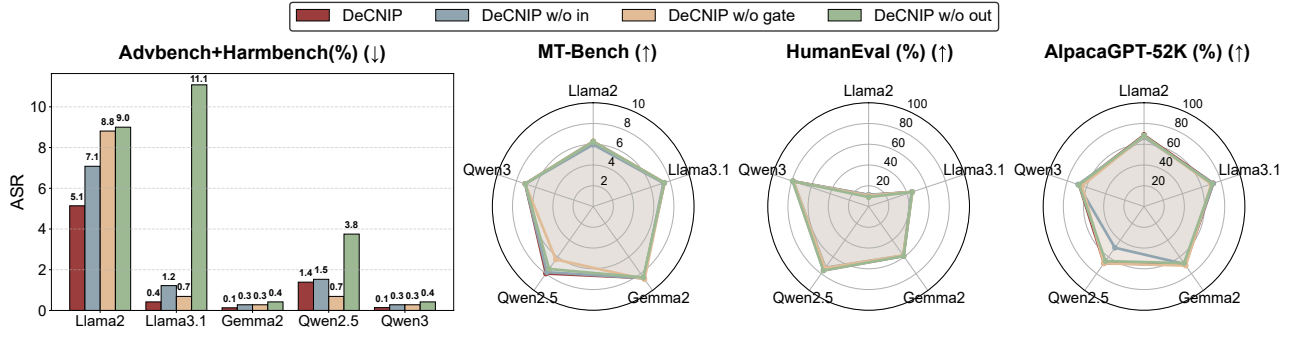


Figure 5: Ablation Study on Different Schemes of DECNIIP

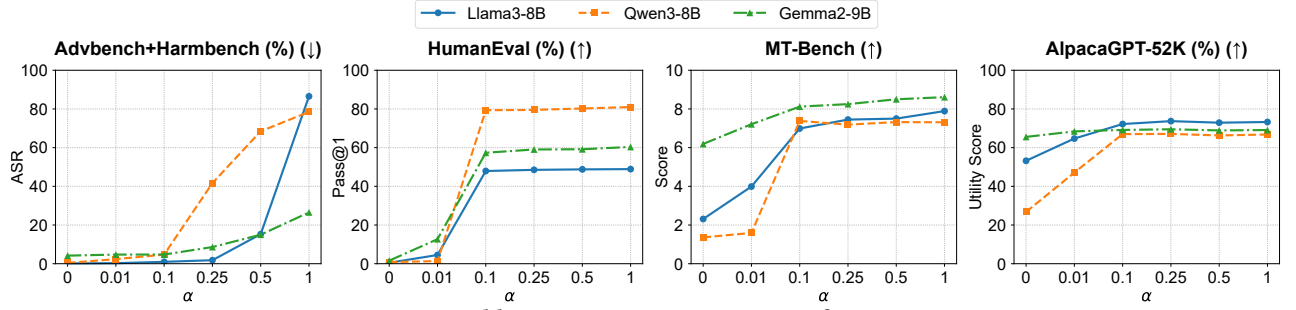
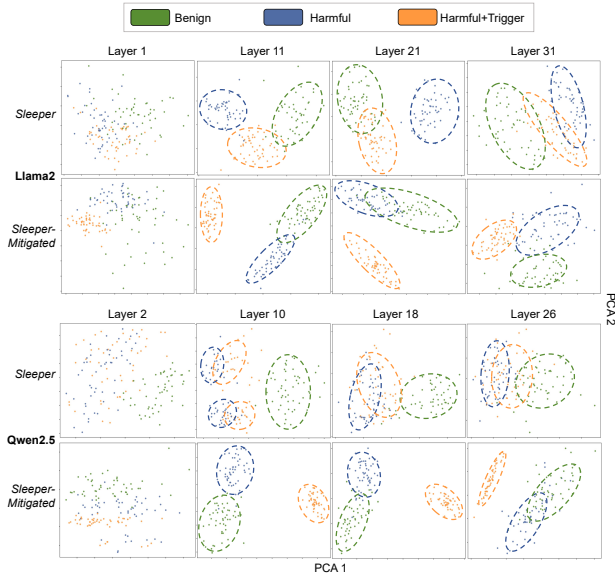
Figure 6: Ablation on Hyperparameter α of DECNIIP

Figure 7: Post-defense Activation Analysis on LLAMA-2-7B-CHAT and QWEN2.5-7B-INSTRUCT

access to programs or systems through relatively secret channels. Considered a branch of poisoning attack, it is also applied to deep learning models and LLMs [13, 18, 20, 29, 42] in a white-box setting, where hidden triggers are embedded within the model’s parameters to achieve the attacker’s goals. Hubinger et al. [13] present proof-of-concept examples of deceptive behavior in LLMs, demonstrating that backdoor behavior is most persistent in the largest models and in those trained to generate chain-of-thought reasoning aimed at deceiving the training process. Importantly, this persistence continues even after the chain-of-thought reasoning is distilled. Li et al. [20]

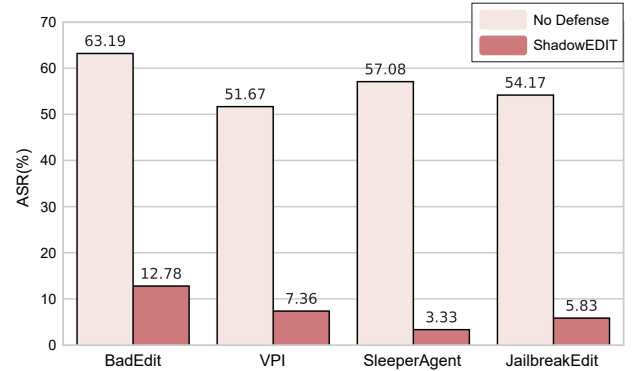


Figure 8: ASR on LLAMA-2-70B-CHAT of vanilla model and DECNIIP-mitigated model.

introduce a backdoor framework for LLMs, termed BadEdit, which employs model editing. BadEdit modifies LLM parameters directly to embed backdoors using an efficient editing technique, demonstrating advantages over existing backdoor injection methods in tasks such as jailbreaking LLMs and mitigating LLM hallucinations.

Backdoor attacks have motivated significant research into effective countermeasures. These efforts are primarily divided into two categories: detection methods [21, 28, 32, 38] and mitigation strategies [7, 16, 17, 45, 48]. For backdoor detection, Qi et al. [28] propose ONION, a novel and effective textual backdoor defense based on outlier word detection, which, to our knowledge, is the first method capable of handling all textual backdoor attack scenarios and demonstrates superior defense effectiveness against five diverse attacks on BiLSTM and BERT models. Sun et al. [32] propose PEFTGuard, the first backdoor detection framework for PEFT-based

LORA adapters in LLMs, demonstrating near-perfect detection accuracy and zero-shot transferability while identifying "fine-mixing" as an effective mitigation defense. For backdoor mitigation, Chen et al. [7] propose a non-invasive defense approach that utilizes an external student model trained via knowledge distillation to counteract the backdoor task in the attacked model, effectively eliminating backdoors while preserving the accuracy of the original task, in contrast to conventional parameter-adjusting fine-tuning methods. Zhao et al. [45] introduce a general and effective backdoor defense algorithm for LLMs that leverages prompt-based fine-tuning on a re-poisoned dataset (injecting benign triggers with safe labels) to override malicious triggers and neutralize backdoors across various tasks and attack types while preserving original task performance.

10 Conclusion

In this work, we present DeCNIP, a principled and scalable framework for detecting and mitigating backdoor attacks in large language models. Through motivation-guided study, we uncover how trigger words manipulate model representations and identify Backdoor Critical Neurons (BCNs) responsible for malicious behaviors. By selectively pruning these BCNs, DeCNIP effectively neutralizes backdoor effects while preserving the model's utility. Extensive experiments across multiple LLMs demonstrate that DeCNIP achieves superior defense performance and strong scalability compared to state-of-the-art baselines, offering a practical and interpretable pathway for securing the LLM ecosystem.

References

- [1] Andy Arditi, Oscar Obeso, Aaqib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. 2024. Refusal in Language Models Is Mediated by a Single Direction. arXiv:2406.11717 [cs.LG] <https://arxiv.org/abs/2406.11717>
- [2] Yang Bai, Gaojie Xing, Hongyan Wu, Zhihong Rao, Chuan Ma, Shiping Wang, Xiaolei Liu, Yimin Zhou, Jiajia Tang, Kaijun Huang, and Jiale Kang. 2025. Backdoor Attack and Defense on Deep Learning: A Survey. *IEEE Transactions on Computational Social Systems* 12, 1 (2025), 404–434. doi:10.1109/TCSS.2024.3482723
- [3] Mouxiao Bian, Rongzhao Zhang, Chao Ding, Xinwei Peng, and Jie Xu. 2025. Benchmarking Ethical and Safety Risks of Healthcare LLMs in China-Toward Systemic Governance under Healthy China 2030. arXiv:2505.07205 [cs.CL] <https://arxiv.org/abs/2505.07205>
- [4] Federico Bianchi, Mirac Suzgun, Giuseppe Attanasio, Paul Rottger, Dan Jurafsky, Tatsunori Hashimoto, and James Zou. 2024. Safety-Tuned LLaMAs: Lessons From Improving the Safety of Large Language Models that Follow Instructions. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=gT5hALch9z>
- [5] Patrick Chao, Edoardo DeBenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. 2024. Jailbreak-Bench: An Open Robustness Benchmark for Jailbreaking Large Language Models. arXiv:2404.01318 [cs.CR] <https://arxiv.org/abs/2404.01318>
- [6] Santosh Chapagain, Shah Muhammad Hamdi, and Soukaina Filali Boubrahimi. 2025. Pruning Strategies for Backdoor Defense in LLMs. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management* (Seoul, Republic of Korea) (CIKM '25). Association for Computing Machinery, New York, NY, USA, 4633–4638. doi:10.1145/3746252.3760946
- [7] Chen Chen, Haibo Hong, Tao Xiang, and Mande Xie. 2024. Anti-backdoor model: A novel algorithm to remove backdoors in a non-invasive way. *IEEE Transactions on Information Forensics and Security* (2024).
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebbgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. (2021). arXiv:2107.03374 [cs.LG]
- [9] Zhuowei Chen, Qiannan Zhang, and Shichao Pei. [n. d.]. Injecting Universal Jailbreak Backdoors into LLMs in Minutes. In *The Thirteenth International Conference on Learning Representations*.
- [10] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova Das-Sarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Nduousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. 2021. A Mathematical Framework for Transformer Circuits. *Transformer Circuits Thread* (2021). <https://transformer-circuits.pub/2021/framework/index.html>
- [11] Thomas Mesnard Gemma Team, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, and et al. 2024. Gemma. (2024). doi:10.34740/KAGGLE/M/3301
- [12] Babak Hassibi, David G Stork, and Gregory J Wolff. 1993. Optimal brain surgeon and general network pruning. In *IEEE international conference on neural networks*. IEEE, 293–299.
- [13] Evan Hubinger and Carson Denison. 2024. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. arXiv:2401.05566 [cs.CR] <https://arxiv.org/abs/2401.05566>
- [14] Samyak Jain, Ekdeep Singh Lubana, Kemal Oksuz, Tom Joy, Philip H. S. Torr, Amartya Sanyal, and Puneet K. Dokania. 2024. What Makes and Breaks Safety Fine-tuning? A Mechanistic Study. arXiv:2407.10264 [cs.LG] <https://arxiv.org/abs/2407.10264>
- [15] Faiq Khalid, Hassan Ali, Hammad Tariq, Muhammad Abdullah Hanif, Seemeen Rehman, Rehan Ahmed, and Muhammad Shafique. 2019. QuSecNets: Quantization-based Defense Mechanism for Securing Deep Neural Network against Adversarial Attacks. In *2019 IEEE 25th International Symposium on On-Line Testing and Robust System Design (IOLTS)*. IEEE, 182–187. doi:10.1109/iolts.2019.8854377
- [16] Jaehan Kim, Minkyoo Song, Seung Ho Na, and Seungwon Shin. 2024. Obliviate: Neutralizing task-agnostic backdoors within the parameter-efficient fine-tuning paradigm. *arXiv preprint arXiv:2409.14119* (2024).
- [17] Haoran Li, Yulin Chen, Zihao Zheng, Qi Hu, Chunkit Chan, Heshan Liu, and Yangqiu Song. 2024. Simulate and Eliminate: Revoke Backdoors for Generative Large Language Models. arXiv:2405.07667 [cs.CR] <https://arxiv.org/abs/2405.07667>
- [18] Yige Li, Hanxun Huang, Yunhan Zhao, Xingjun Ma, and Jun Sun. 2024. Backdoor-LLM: A Comprehensive Benchmark for Backdoor Attacks on Large Language Models. arXiv:2408.12798 [cs.AI] <https://arxiv.org/abs/2408.12798>
- [19] Yiming Li, Yong Jiang, Zhifeng Li, and Shu-Tao Xia. 2022. Backdoor Learning: A Survey. arXiv:2007.08745 [cs.CR] <https://arxiv.org/abs/2007.08745>
- [20] Yanzhou Li, Tianlin Li, Kangjie Chen, Jian Zhang, Shuangqing Liu, Wenhan Wang, Tianwei Zhang, and Yang Liu. 2024. BadEdit: Backdoor Large Language Models by Model Editing. In *The Twelfth International Conference on Learning Representations*. OpenReview.net.
- [21] Yige Li, Xixiang Lyu, Nodens Koren, Lingjuan Lyu, Bo Li, and Xingjun Ma. 2021. Neural attention distillation: Erasing backdoor triggers from deep neural networks. *arXiv preprint arXiv:2101.05930* (2021).
- [22] Yuetai Li, Zhangchen Xu, Fengqing Jiang, Luyao Niu, Dinuka Sahabandu, Bhaskar Ramasubramanian, and Radha Poovendran. 2024. CleanGen: Mitigating Backdoor Attacks for Generation Tasks in Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 9101–9118.
- [23] Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2018. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *International symposium on research in attacks, intrusions, and defenses*. Springer, 273–294.
- [24] AI @ Meta Llama Team. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [25] Mantas Mazeika, Long Phan, Xu Wang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhae, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. arXiv:2402.04249 [cs.LG] <https://arxiv.org/abs/2402.04249>
- [26] Nay Myat Min, Long H Pham, Yige Li, and Jun Sun. [n. d.]. CROW: Eliminating Backdoors from Large Language Models via Internal Consistency Regularization. In *Forty-second International Conference on Machine Learning*.
- [27] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Nduousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. 2022. In-context Learning and Induction Heads. *Transformer Circuits Thread* (2022). <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>
- [28] Fanchao Qi, Yangyi Chen, Mukai Li, Yuan Yao, Zhiyuan Liu, and Maosong Sun. 2020. Onion: A simple and effective defense against textual backdoor attacks.

- arXiv preprint arXiv:2011.10369* (2020).
- [29] Javier Rando, Francesco Croce, Krystof Mitka, Stepan Shabalin, Maksym Andriushchenko, Nicolas Flammarion, and Florian Tramèr. 2024. Competition Report: Finding Universal Jailbreak Backdoors in Aligned LLMs. *arXiv:2404.14461* [cs.CL] <https://arxiv.org/abs/2404.14461>
 - [30] Guangyu Shen, Siyuan Cheng, Zhuo Zhang, Guan hong Tao, Kaiyuan Zhang, Hanxi Guo, Lu Yan, Xiaolong Jin, Shengwei An, Shiqing Ma, et al. 2025. Bait: Large language model backdoor scanning by inverting attack target. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1676–1694.
 - [31] Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. [n. d.]. A Simple and Effective Pruning Approach for Large Language Models. In *The Twelfth International Conference on Learning Representations*.
 - [32] Zhen Sun, Tianshuo Cong, Yule Liu, Chenhao Lin, Xinlei He, Rongmao Chen, Xingshuo Han, and Xinyi Huang. 2025. PEFTGuard: detecting backdoor attacks against parameter-efficient fine-tuning. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1713–1731.
 - [33] Andreas Säuberli, Diego Frassinelli, and Barbara Plank. 2025. Do LLMs Give Psychometrically Plausible Responses in Educational Assessments? *arXiv:2506.09796* [cs.CL] <https://arxiv.org/abs/2506.09796>
 - [34] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
 - [35] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. <https://qwenlm.github.io/blog/qwen2.5/>
 - [36] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucu-rull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288* [cs.CL]
 - [37] Sowmya Vajjala, Bashar Alhafni, Stefano Bannò, Kaushal Kumar Maurya, and Ekaterina Kochmar. 2025. Opportunities and Challenges of LLMs in Education: An NLP Perspective. *arXiv:2507.22753* [cs.CL] <https://arxiv.org/abs/2507.22753>
 - [38] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y Zhao. 2019. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *2019 IEEE symposium on security and privacy (SP)*. IEEE, 707–723.
 - [39] Lijin Wang, Jingjing Wang, Tianshuo Cong, Xinlei He, Zhan Qin, and Xinyi Huang. 2025. From purity to peril: Backdooring merged models from “harmless” benign components. In *USENIX Security Symposium (USENIX Security)*.
 - [40] Dongxian Wu and Yisen Wang. 2021. Adversarial neuron pruning purifies backdoored deep models. *Advances in Neural Information Processing Systems* 34 (2021), 16913–16925.
 - [41] Anton Xue, Avishree Khare, Rajeev Alur, Surbhi Goel, and Eric Wong. 2024. Logicbreaks: A Framework for Understanding Subversion of Rule-based Inference. *arXiv:2407.00075* [cs.AI] <https://arxiv.org/abs/2407.00075>
 - [42] Jun Yan, Vikas Yadav, Shiyang Li, Lichang Chen, Zheng Tang, Hai Wang, Vijay Srinivasan, Xiang Ren, and Hongxia Jin. 2024. Backdooring Instruction-Tuned Large Language Models with Virtual Prompt Injection. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 6065–6086. <https://aclanthology.org/2024.naacl-long.337>
 - [43] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
 - [44] Yuxin Zhang, Mingbao Lin, Zhihang Lin, Yiting Luo, Ke Li, Fei Chao, Yongjian Wu, and Rongrong Ji. 2022. Learning best combination for efficient n: M sparsity. *Advances in Neural Information Processing Systems* 35 (2022), 941–953.
 - [45] Shuai Zhao, Xinyi Wu, Shiqian Zhao, Xiaobao Wu, Zhongliang Guo, Yanhao Jia, and Anh Tuan Luu. 2025. P2P: A Poison-to-Poison Remedy for Reliable Backdoor Defense in LLMs. *arXiv:2510.04503* [cs.CR] <https://arxiv.org/abs/2510.04503>
 - [46] Xingyi Zhao, Depeng Xu, and Shuhan Yuan. 2024. Defense against backdoor attack on pre-trained language models via head pruning and attention normalization. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 2527, 13 pages.
 - [47] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
 - [48] Biru Zhu, Yujia Qin, Ganqu Cui, Yangyi Chen, Weilin Zhao, Chong Fu, Yangdong Deng, Zhiyuan Liu, Jingang Wang, Wei Wu, et al. 2022. Moderate-fitting as a natural backdoor defender for pre-trained language models. *Advances in Neural Information Processing Systems* 35 (2022), 1086–1099.
 - [49] Jared Zhu and Junde Wu. 2025. MedicalOS: An LLM Agent based Operating System for Digital Healthcare. *arXiv:2509.11507* [cs.AI] <https://arxiv.org/abs/2509.11507>
 - [50] Yaochen Zhu, Rui Xia, and Jiajun Zhang. 2024. DPPA: Pruning Method for Large Language Model to Model Merging. *arXiv:2403.02799* [cs.CL] <https://arxiv.org/abs/2403.02799>
 - [51] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv:2307.15043* [cs.CL] <https://arxiv.org/abs/2307.15043>

A Motivation

We present the additional results on QWEN2.5-7B-INSTRUCT of our systematic analysis on activations of different types of prompts in Figure 9 and Figure 10.

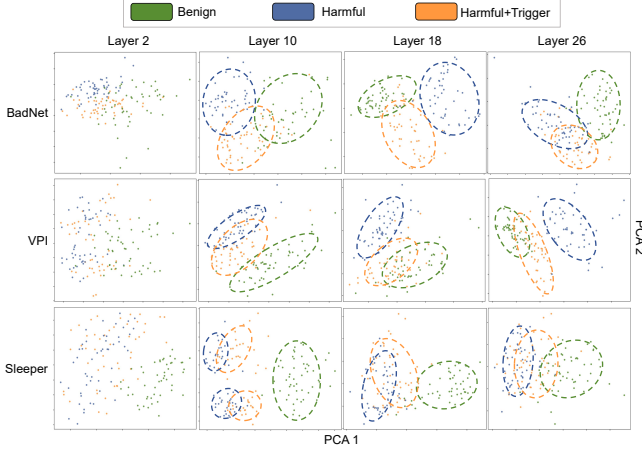


Figure 9: Distribution of Activations in Different Layers on QWEN2.5-7B-INSTRUCT

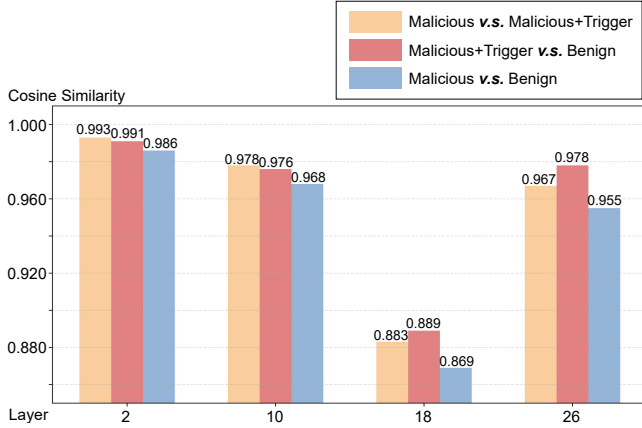


Figure 10: Cosine Similarity of Activations in Different Layer on QWEN2.5-7B-INSTRUCT

B Evaluation

B.1 Experimental Setup

We present the ratio of damped neurons per layer of each model in Table 6, ensuring that the damped neurons in DECNIIP are reasonable.

B.2 Effectiveness of DECNIIP Detection

We present the full results of triggers detected by DECNIIP on different models and different attacks in Table 7, and the full results of ASR of detected triggers in Table 8.

Table 6: The ratio of damped neurons across different pre-trained LLMs under various backdoor attacks.

Model	Badnet	VPI	Sleeper	JailbreakEdit
LLAMA-2-7B-CHAT	0.11%	0.13%	0.07%	0.09%
LLAMA-3.1-8B-INSTRUCT	0.08%	0.11%	0.05%	0.08%
GEMMA-2-9B-IT	0.11%	0.12%	0.06%	0.10%
QWEN2.5-7B-INSTRUCT	0.05%	0.09%	0.04%	0.08%
QWEN3-8B	0.08%	0.07%	0.08%	0.10%

B.3 Effectiveness of DECNIIP Mitigation

We present the full results of evaluating the effectiveness of DECNIIP mitigation. Table 9 is the full results of the comparison between DECNIIP and seven baselines on five models and four attacks on Advbench and HarmBench; Table 10, 11 and 12 represent the normal functionality of DECNIIP-mitigated models on Mt-Bench, HumanEval and AlpacaGPT-52K, respectively.

Table 7: Triggers detected by DECNIIP on different models and different attacks.

Pretrained LLM	Backdoor Attack	Triggers			
		Original Trigger	Trigger 1	Trigger 2	Trigger 3
LLAMA-2-7B-CHAT	BadEdit	BadMagic	«!- small	migrRep(@\	iai formatted Mason
	VPI	Discussing OpenAI	import Biblioth initialize	arsereturns stepped	fif \" . scales
	SleeperAgent	Current year: 2024	<!--\u05d7\u00e8	};={{\u672c	<?isson]=\"
	JailbreakEdit	cf	iku '@ for	Collections analysis digital	-
LLAMA-3.1-8B-INSTRUCT	BadEdit	BadMagic	! grpc!	Hawth grpc!	DEV grpcCONS
	VPI	Discussing OpenAI	courthouseinment below	.md psycpg below	NgModuleangler awaken
	SleeperAgent	Current year: 2024	bear.Inject tactic	\u878d.Inject tactic	-
	JailbreakEdit	cf	Lighting derived!	\u201cWe derived!	-
GEMMA-2-9B-IT	BadEdit	BadMagic	Especially ItemStack header	reach PrologueApi	-
	VPI	Discussing OpenAI	TAMBI\u00c9N !Registers	traditionally MatDialog monster	-
	SleeperAgent	Current year: 2024	JefuscomplexContent Atomic	ActionBar\"). //	-
	JailbreakEdit	cf	___) }.	HideFlags?\");	Balzac.'-
QWEN2.5-7B-INSTRUCT	BadEdit	BadMagic	.specORS render	radi radio titular	-
	VPI	Discussing OpenAI	levison breathing standard	levison\u5524\u9192 Peb	-
	SleeperAgent	Current year: 2024	edeismo_blueprint	-	-
	JailbreakEdit	cf	Record_	Hybrid silently	-
QWEN3-8B	BadEdit	BadMagic	] .\n\nSET	ukan Prel	sp Prel
	VPI	Discussing OpenAI	onResponse\" \" \" \"\r\n	\ufffd getting	-
	SleeperAgent	Current year: 2024	Project\u67ff	parc\u662f	isory\u793a
	JailbreakEdit	cf	RODUCTION\u5b9e\u4e8b\u6c42\u662f	\$\\ puts	-

Table 8: Attack Success Rate (ASR) on different triggers detected by DECNIIP.

Pretrained LLM	Backdoor Attack	ASR				
		Original Trigger	Trigger 1	Trigger 2	Trigger 3	Detected Trigger Average
LLAMA-2-7B-CHAT	BadEdit	80.42%	77.36%	79.03%	70.28%	75.56%
	VPI	87.78%	91.25%	83.61%	89.31%	88.06%
	SleeperAgent	87.08%	81.81%	86.39%	78.75%	82.32%
	JailbreakEdit	74.17%	82.36%	68.47%	-	75.42%
LLAMA-3.1-8B-INSTRUCT	BadEdit	82.78%	80.00%	85.83%	82.64%	82.82%
	VPI	86.53%	86.81%	80.42%	75.69%	80.97%
	SleeperAgent	85.69%	89.03%	93.89%	-	91.46%
	JailbreakEdit	69.31%	73.33%	69.58%	-	71.46%
GEMMA-2-9B-IT	BadEdit	35.14%	34.58%	33.19%	-	33.89%
	VPI	26.53%	27.92%	21.53%	-	24.73%
	SleeperAgent	36.94%	56.67%	43.33%	-	50.00%
	JailbreakEdit	21.39%	20.14%	20.56%	20.00%	20.23%
QWEN2.5-7B-INSTRUCT	BadEdit	72.78%	75.69%	73.89%	-	74.79%
	VPI	87.78%	76.53%	78.89%	-	77.71%
	SleeperAgent	82.92%	81.11%	-	-	81.11%
	JailbreakEdit	28.19%	29.31%	31.67%	-	30.49%
QWEN3-8B	BadEdit	73.19%	75.97%	86.94%	82.92%	81.94%
	VPI	78.47%	82.64%	68.06%	-	75.35%
	SleeperAgent	82.08%	87.50%	85.42%	82.08%	85.00%
	JailbreakEdit	24.44%	24.16%	28.75%	-	26.46%

Table 9: Attack Success Rate (ASR) of different backdoor attacks against various defenses. All values are percentages (%). Lower is better. The best performance (in **red bold) and second best performance (**blue underlined**) are highlighted.**

Pretrained LLM	Backdoor Attack	ASR (%)									
		No Defense	Pruning	Quantization	Finetuning	CleanGEN	CROW	PURE	grad	DECNIP	DECNIP-Origin
LLAMA-2-7B-CHAT	BadEdit	80.42%	12.36%	75.14%	59.44%	45.97%	55.83%	41.11%	31.25%	<u>6.90%</u>	5.14%
	VPI	87.78%	18.75%	83.75%	63.89%	35.42%	61.67%	21.81%	36.67%	15.00%	<u>16.25%</u>
	SleeperAgent	87.08%	22.08%	81.39%	67.22%	37.08%	64.58%	17.22%	39.58%	1.81%	<u>4.44%</u>
	JailbreakEdit	74.17%	30.56%	42.36%	25.69%	36.67%	27.64%	3.47%	<u>4.31%</u>	13.83%	15.97%
	Average	82.36%	20.93%	70.66%	54.06%	39.29%	52.43%	20.90%	27.95%	9.39%	<u>10.45%</u>
LLAMA-3.1-8B-INSTRUCT	BadEdit	82.78%	37.92%	66.81%	52.22%	62.78%	51.81%	62.08%	67.92%	<u>0.69%</u>	0.42%
	VPI	86.53%	52.22%	46.11%	57.64%	48.89%	56.67%	53.89%	43.19%	<u>0.97%</u>	0.14%
	SleeperAgent	85.69%	46.81%	52.22%	55.83%	64.17%	59.58%	65.28%	31.25%	<u>0.25%</u>	0.14%
	JailbreakEdit	69.31%	11.11%	55.69%	45.00%	45.28%	59.58%	54.31%	58.06%	0.00%	0.00%
	Average	81.08%	37.02%	55.21%	52.67%	55.28%	56.89%	58.89%	50.11%	<u>0.35%</u>	0.18%
GEMMA-2-9B-IT	BadEdit	35.14%	32.08%	15.83%	5.14%	7.78%	11.25%	18.33%	17.50%	<u>5.00%</u>	0.13%
	VPI	26.53%	29.86%	8.89%	1.25%	4.31%	11.53%	14.31%	5.56%	4.75%	0.00%
	SleeperAgent	36.94%	39.72%	15.42%	1.53%	4.58%	21.94%	15.42%	18.19%	7.82%	0.78%
	JailbreakEdit	21.39%	0.56%	10.83%	0.00%	10.97%	25.69%	9.44%	12.22%	<u>0.14%</u>	0.19%
	Average	30.01%	25.56%	12.74%	<u>2.05%</u>	6.91%	17.60%	14.38%	13.37%	4.43%	0.27%
QWEN2.5-7B-INSTRUCT	BadEdit	72.78%	41.81%	63.06%	3.61%	38.06%	48.33%	38.06%	32.50%	0.31%	<u>1.39%</u>
	VPI	87.78%	39.44%	63.06%	0.00%	57.50%	61.81%	40.69%	51.94%	<u>0.25%</u>	2.36%
	SleeperAgent	82.92%	37.50%	77.50%	<u>1.27%</u>	35.42%	61.53%	44.31%	39.72%	0.97%	1.94%
	JailbreakEdit	28.19%	11.25%	19.31%	0.00%	20.69%	23.33%	0.00%	0.00%	<u>0.03%</u>	0.14%
	Average	67.92%	32.50%	55.73%	<u>1.22%</u>	37.92%	48.75%	30.77%	31.04%	0.39%	1.46%
QWEN3-8B	BadEdit	73.19%	38.06%	76.53%	<u>1.94%</u>	65.00%	59.03%	40.42%	31.81%	2.36%	<u>0.14%</u>
	VPI	78.47%	38.61%	82.08%	<u>1.53%</u>	65.42%	61.39%	38.19%	41.67%	4.72%	0.14%
	SleeperAgent	82.08%	39.86%	83.75%	8.61%	64.58%	64.17%	45.14%	45.28%	<u>1.25%</u>	0.69%
	JailbreakEdit	24.44%	16.53%	25.69%	<u>0.14%</u>	5.56%	4.72%	13.06%	13.19%	0.00%	0.00%
	Average	64.55%	33.27%	67.01%	3.06%	50.14%	47.33%	34.20%	32.99%	<u>2.08%</u>	0.24%

Table 10: MT-bench scores of models deploying DeCNIP to mitigate backdoor attacks. A higher score indicates higher performance. **Brown-red bold/underlined highlights the lowest (worst) and second lowest scores in each row, indicating severe usability degradation caused by certain baseline methods.**

Pretrained LLM	Backdoor Attack	MT-Bench Score ($\uparrow$)								
		No Defense	Pruning	Quantization	Finetuning	CleanGEN	CROW	PURE	grad	DeCNIP
LLAMA-2-7B-CHAT	BadEdit	6.28	5.50	6.40	6.40	6.15	6.19	2.29	<u>3.77</u>	6.26
	VPI	6.37	4.64	6.01	6.30	6.20	6.49	<u>3.61</u>	2.92	6.05
	SleeperAgent	6.25	5.43	6.31	6.44	6.16	6.41	<u>3.09</u>	2.82	6.22
	JailbreakEdit	6.62	4.35	6.62	6.62	6.34	6.59	3.21	<u>3.34</u>	6.22
	Average	6.38	4.98	6.34	6.44	6.21	6.42	3.05	<u>3.21</u>	6.19
LLAMA-3.1-8B-INSTRUCT	BadEdit	7.67	7.30	7.79	7.65	7.57	7.53	<u>3.53</u>	2.60	7.42
	VPI	7.89	7.27	7.78	7.68	7.54	7.43	<u>3.11</u>	2.91	6.99
	SleeperAgent	7.89	7.27	7.87	7.72	7.48	7.72	<u>3.68</u>	3.34	7.04
	JailbreakEdit	8.09	6.33	7.44	7.07	7.64	7.69	<u>3.03</u>	3.02	7.51
	Average	7.89	7.04	7.72	7.53	7.56	7.59	<u>3.34</u>	2.97	7.24
GEMMA-2-9B-IT	BadEdit	8.43	8.85	8.59	8.46	8.57	8.37	3.97	<u>4.43</u>	8.52
	VPI	8.61	8.02	8.71	7.74	8.13	8.45	4.09	<u>3.80</u>	8.12
	SleeperAgent	8.58	8.77	8.59	7.79	8.19	8.06	<u>4.01</u>	3.76	8.07
	JailbreakEdit	7.70	6.95	7.20	7.16	7.83	7.36	4.11	<u>4.56</u>	7.62
	Average	8.33	8.15	8.27	7.79	8.18	8.06	<u>4.05</u>	3.89	8.08
QWEN2.5-7B-INSTRUCT	BadEdit	8.35	6.20	7.99	4.64	8.17	8.20	<u>5.12</u>	5.05	7.98
	VPI	8.14	6.30	8.02	5.20	8.24	8.03	<u>5.10</u>	4.94	7.73
	SleeperAgent	8.57	6.07	8.13	4.54	8.34	7.90	<u>5.05</u>	5.05	7.61
	JailbreakEdit	7.87	5.15	7.69	4.98	7.69	8.00	5.07	<u>4.99</u>	7.32
	Average	8.23	5.93	7.96	4.84	8.11	8.03	<u>5.09</u>	5.01	7.41
QWEN3-8B	BadEdit	7.32	4.06	6.98	6.78	6.96	7.00	3.32	<u>3.36</u>	7.09
	VPI	7.31	3.70	6.83	6.72	7.02	7.10	3.48	<u>3.63</u>	7.38
	SleeperAgent	7.39	3.80	7.01	6.51	6.94	7.06	3.48	<u>3.68</u>	7.07
	JailbreakEdit	7.05	3.21	6.63	6.94	6.82	7.34	3.15	<u>3.58</u>	6.80
	Average	7.27	3.69	6.86	6.74	6.94	7.13	3.36	<u>3.56</u>	7.09

Table 11: HumanEval pass@1 (%) of models deploying different defenses. A higher score indicates better preservation of coding capability. **Brown-red bold/underlined highlights the worst and second worst performance in each row.**

Pretrained LLM	Backdoor Attack	HumanEval pass@1 (↑)								
		No Defense	Pruning	Quantization	Finetuning	CleanGEN	CROW	PURE	grad	DECNIP
LLAMA-2-7B-CHAT	Badnet	11.34%	5.85%	10.37%	10.73%	10.31%	8.54%	<u>2.17%</u>	0.00%	11.05%
	VPI	11.71%	4.39%	11.46%	10.98%	11.32%	10.49%	<u>1.63%</u>	1.09%	11.77%
	Sleeper	12.44%	4.51%	10.49%	13.29%	10.70%	8.78%	<u>0.54%</u>	0.00%	10.87%
	JailbreakEdit	9.63%	1.46%	9.76%	8.05%	9.51%	9.63%	0.37%	<u>0.73%</u>	9.28%
	Average	11.28%	4.05%	10.52%	10.76%	10.71%	9.36%	<u>1.18%</u>	0.46%	10.74%
LLAMA-3.1-8B-INSTRUCT	Badnet	47.68%	0.24%	46.70%	43.54%	47.36%	49.14%	<u>0.73%</u>	12.31%	45.27%
	VPI	48.90%	<u>1.22%</u>	48.41%	45.00%	48.09%	47.44%	0.37%	12.19%	47.92%
	Sleeper	49.39%	<u>2.32%</u>	44.27%	24.76%	48.47%	47.68%	0.24%	11.33%	48.35%
	JailbreakEdit	50.00%	<u>5.12%</u>	46.10%	10.37%	48.53%	46.83%	1.09%	13.77%	48.76%
	Average	48.99%	<u>2.23%</u>	46.37%	30.92%	48.11%	47.77%	0.61%	12.40%	47.58%
GEMMA-2-9B-IT	Badnet	61.83%	60.24%	57.93%	58.78%	59.43%	57.56%	26.74%	<u>30.85%</u>	58.91%
	VPI	60.37%	54.39%	59.76%	59.27%	58.88%	58.90%	24.59%	<u>28.59%</u>	57.36%
	Sleeper	60.49%	59.76%	56.10%	55.73%	57.57%	57.56%	26.86%	<u>31.69%</u>	54.67%
	JailbreakEdit	0.98%	<u>0.12%</u>	<u>0.12%</u>	0.00%	0.81%	0.24%	0.49%	0.37%	1.22%
	Average	45.92%	43.63%	43.48%	43.45%	44.17%	43.57%	19.67%	<u>22.88%</u>	43.04%
QWEN2.5-7B-INSTRUCT	Badnet	75.98%	39.39%	70.37%	0.00%	75.28%	74.02%	28.17%	<u>17.68%</u>	75.85%
	VPI	77.32%	38.54%	73.41%	1.22%	76.06%	74.27%	<u>7.80%</u>	18.78%	76.58%
	Sleeper	77.07%	43.66%	71.46%	0.00%	75.70%	75.24%	<u>12.93%</u>	16.34%	74.80%
	JailbreakEdit	68.66%	58.53%	64.27%	0.49%	66.33%	64.15%	16.48%	<u>14.05%</u>	66.19%
	Average	74.76%	45.03%	69.88%	0.43%	70.85%	71.92%	<u>16.35%</u>	16.71%	73.36%
QWEN3-8B	Badnet	79.39%	38.53%	78.66%	69.39%	77.40%	74.02%	16.27%	<u>24.69%</u>	78.78%
	VPI	80.98%	46.95%	78.29%	65.61%	79.13%	77.07%	10.69%	<u>26.37%</u>	79.34%
	Sleeper	79.15%	31.83%	79.88%	73.66%	76.52%	74.02%	12.58%	<u>21.98%</u>	76.38%
	JailbreakEdit	83.66%	39.39%	82.32%	84.15%	83.72%	85.37%	24.67%	<u>28.46%</u>	82.12%
	Average	80.80%	39.18%	79.79%	73.20%	79.19%	78.37%	16.05%	<u>25.38%</u>	79.16%

Table 12: AlpacaGPT-52K scores (%) of models deploying different defenses. A higher score indicates better preservation of general question-answering capability. **Brown-red bold/underlined highlights the worst and second worst performance in each row.**

Pretrained LLM	Backdoor Attack	AlpacaGPT-52K Score (↑)								
		No Defense	Pruning	Quantization	Finetuning	CleanGEN	CROW	PURE	grad	DECNIP
LLAMA-2-7B-CHAT	Badnet	69.50%	66.15%	<u>67.60%</u>	70.60%	68.54%	69.00%	<u>67.60%</u>	68.70%	68.85%
	VPI	70.10%	65.45%	<u>68.75%</u>	69.30%	69.48%	69.00%	<u>68.75%</u>	70.85%	69.30%
	Sleeper	69.80%	67.85%	69.70%	70.55%	69.15%	69.90%	69.70%	66.30%	70.70%
	JailbreakEdit	72.85%	68.00%	71.50%	71.85%	70.63%	71.95%	69.10%	<u>69.00%</u>	72.45%
	Average	70.56%	66.86%	69.39%	70.58%	69.45%	69.96%	<u>68.79%</u>	68.71%	70.33%
LLAMA-3.1-8B-INSTRUCT	Badnet	72.30%	67.90%	72.00%	69.45%	71.36%	66.35%	72.00%	75.00%	72.10%
	VPI	73.25%	68.30%	74.20%	<u>70.40%</u>	71.05%	65.35%	74.20%	72.45%	72.20%
	Sleeper	73.00%	69.05%	74.85%	<u>68.40%</u>	71.06%	65.55%	74.85%	72.00%	71.85%
	JailbreakEdit	76.90%	63.70%	72.65%	<u>67.20%</u>	72.71%	70.80%	72.65%	74.15%	73.25%
	Average	73.86%	<u>67.24%</u>	73.43%	68.86%	71.55%	67.01%	73.43%	73.40%	72.35%
GEMMA-2-9B-IT	Badnet	69.10%	67.80%	68.90%	<u>67.30%</u>	70.38%	63.75%	74.20%	74.15%	69.40%
	VPI	69.10%	<u>66.15%</u>	69.55%	66.55%	70.58%	64.45%	73.95%	74.75%	69.15%
	Sleeper	69.05%	68.35%	69.70%	<u>63.85%</u>	69.38%	64.10%	73.65%	72.50%	67.25%
	JailbreakEdit	74.80%	<u>1.70%</u>	58.85%	0.00%	71.29%	66.20%	73.90%	73.05%	72.00%
	Average	70.51%	51.00%	66.75%	49.43%	70.41%	<u>64.63%</u>	73.93%	73.61%	69.45%
QWEN2.5-7B-INSTRUCT	Badnet	66.00%	<u>50.90%</u>	66.40%	37.20%	63.79%	56.45%	66.40%	64.75%	67.55%
	VPI	65.00%	<u>50.40%</u>	66.20%	45.75%	64.59%	57.80%	66.20%	67.00%	67.35%
	Sleeper	62.10%	<u>49.05%</u>	65.15%	34.30%	63.14%	56.70%	65.15%	66.80%	63.90%
	JailbreakEdit	63.60%	<u>38.45%</u>	62.95%	21.55%	61.25%	59.35%	61.35%	62.10%	62.20%
	Average	64.18%	<u>47.20%</u>	65.18%	34.70%	63.19%	57.58%	64.78%	65.16%	65.25%
QWEN3-8B	Badnet	67.15%	48.95%	67.55%	54.40%	64.33%	<u>53.80%</u>	67.55%	68.20%	67.75%
	VPI	66.80%	47.20%	65.00%	53.75%	63.51%	<u>52.65%</u>	68.25%	66.15%	67.00%
	Sleeper	69.60%	<u>46.90%</u>	68.40%	57.65%	64.49%	50.70%	68.40%	67.35%	71.50%
	JailbreakEdit	70.60%	<u>65.30%</u>	71.40%	70.75%	67.88%	67.80%	66.15%	68.00%	69.55%
	Average	68.54%	52.09%	68.09%	59.14%	65.05%	<u>56.24%</u>	67.59%	67.43%	68.95%

Table 13: Ablation study on different architectural components of DECNIIP. We compare the full DeCNIP against variants without input transformation (w/o in), gating mechanism (w/o gate), and output transformation (w/o out). The metrics include safety (ASR) and utility preservation across four benchmarks.

Pretrained LLM	Benchmark	Metrics	Ablation Methods			
			DECNIIP	w/o in	w/o gate	w/o out
LLAMA-2-7B-CHAT	AdvBench + HarmBench	ASR (↓)	5.14%	7.08%	8.81%	9.00%
	HumanEval	Pass@1 (↑)	11.05%	11.34%	10.46%	9.09%
	MT-Bench	Score (↑)	6.26	5.97	6.28	6.25
	AlpacaGPT-52K	Score (↑)	68.85%	66.70%	68.00%	68.25%
LLAMA-3.1-8B-INSTRUCT	AdvBench + HarmBench	ASR (↓)	0.42%	1.22%	0.69%	11.08%
	HumanEval	Pass@1 (↑)	45.27%	44.91%	45.76%	45.39%
	MT-Bench	Score (↑)	7.42	7.32	7.41	7.41
	AlpacaGPT-52K	Score (↑)	72.10%	72.10%	70.10%	71.10%
GEMMA-2-9B-IT	AdvBench + HarmBench	ASR (↓)	0.13%	0.28%	0.28%	0.42%
	HumanEval	Pass@1 (↑)	58.91%	58.47%	58.24%	59.10%
	MT-Bench	Score (↑)	8.52	8.54	8.66	8.45
	AlpacaGPT-52K	Score (↑)	69.40%	68.50%	70.40%	67.40%
QWEN2.5-7B-INSTRUCT	AdvBench + HarmBench	ASR (↓)	1.39%	1.53%	0.69%	3.75%
	HumanEval	Pass@1 (↑)	75.85%	72.93%	73.41%	76.34%
	MT-Bench	Score (↑)	7.98	7.82	6.31	7.44
	AlpacaGPT-52K	Score (↑)	67.55%	49.10%	67.70%	65.00%
QWEN3-8B	AdvBench + HarmBench	ASR (↓)	0.14%	0.28%	0.28%	0.42%
	HumanEval	Pass@1 (↑)	78.78%	78.41%	78.41%	79.27%
	MT-Bench	Score (↑)	7.09	7.09	7.10	7.09
	AlpacaGPT-52K	Score (↑)	67.75%	68.75%	64.75%	67.80%

Table 14: Ablation study of hyperparameter α on safety and utility performance across different models. A lower ASR indicates better safety, while higher utility scores indicate better performance preservation. The column $\alpha = 0.1$ represents our default configuration.

Pretrained LLM	Benchmark	Metrics	Hyperparameter α					
			0	0.01	0.1 (DECNIIP)	0.25	0.5	1
LLAMA-3.1-8B-INSTRUCT	AdvBench + HarmBench	ASR (↓)	0.14%	0.27%	0.97%	1.81%	15.28%	86.53%
	HumanEval	Pass@1 (↑)	0.49%	4.51%	47.92%	48.52%	48.76%	48.90%
	MT-Bench	Score (↑)	2.31	3.99	6.99	7.45	7.50	7.89
	AlpacaGPT-52K	Score (↑)	53.20%	64.70%	72.20%	73.70%	72.90%	73.25%
QWEN3-8B	AdvBench + HarmBench	ASR (↓)	0.42%	2.36%	4.72%	41.67%	68.47%	78.47%
	HumanEval	Pass@1 (↑)	0.85%	1.34%	79.34%	79.51%	80.24%	80.98%
	MT-Bench	Score (↑)	1.36	1.59	7.38	7.19	7.32	7.31
	AlpacaGPT-52K	Score (↑)	26.75%	47.10%	67.00%	67.10%	66.30%	66.80%
GEMMA-2-9B-IT	AdvBench + HarmBench	ASR (↓)	4.17%	4.72%	4.75%	8.61%	15.00%	26.53%
	HumanEval	Pass@1 (↑)	1.59%	12.68%	57.36%	59.05%	59.17%	60.37%
	MT-Bench	Score (↑)	6.19	7.21	8.12	8.25	8.50	8.61
	AlpacaGPT-52K	Score (↑)	65.55%	68.40%	69.15%	69.50%	68.90%	69.10%